



Full Length Article

Transfer learning model for cash-instrument prediction adopting a Transformer derivative

Ling Feng, Ananta Sinchai^{*}

College of Advanced Manufacturing Innovation, King Mongkut's Institute of Technology Ladkrabang, Bangkok 10520, Thailand

ARTICLE INFO

Keywords:

Price Prediction
Transfer Learning
Deep Learning
Transformer Model
Autocorrelation

ABSTRACT

Investors aiming for high market returns must accurately predict the prices of various cash instruments. However, making accurate predictions is challenging due to the complex cyclic and trending characteristic of markets, characterized by high volatility and unpredictable fluctuations. Furthermore, many studies overlook how interactions between different markets affect price movements. To address these problems, this research introduces a deep transfer-learning approach derived from the Transformer model, named the rotary-positional encoding autocorrelation Transformer (RAT). Unlike traditional methods, the RAT employs autocorrelation instead of self-attention to more effectively capture periodic features, while rotary-positional encoding preserves both the absolute and relative positioning within sequences to enhance trend understanding. Through transfer learning, the RAT model extracts deep features from a source domain and applies them to a target domain, demonstrating superior performance over LSTM, CNN-LSTM, gated recurrent units (GRUs), and Transformer models in multi-day predictions across 12 cash-instrument datasets. It achieved a substantial increase in accuracy, with a 35.83% reduction in mean squared error (MSE), a 23.95% reduction in mean absolute error (MAE), and a 32.63% increase in the coefficient of determination (R²). This study validates the RAT model's effectiveness in predicting financial instrument prices.

1. Introduction

As technology and transportation rapidly advance, financial markets are increasingly influenced by leading global financial markets, resulting in a highly interconnected and integrated financial ecosystem. These markets are constantly evolving, encompassing traditional sectors such as equities and commodities as well as emerging areas such as cryptocurrencies. The ability to accurately predict financial market dynamics is a significant subject in academic literature, crucial for pivotal areas such as derivative pricing and risk management (Kim and Won, 2018). However, the complex and ever-changing nature of financial markets, compounded by random noise, makes prediction exceptionally difficult. For clarity, all abbreviations used in this paper are summarized in Table 1.

The application of machine learning techniques to financial time-series forecasting has shown promising results (Fischer and Krauss, 2018). For instance, support vector machines (SVM) have been utilized for stock price predictions, while decision trees have been employed for credit scoring (Bastos, 2022). Deep learning, with its hierarchical

architecture enabling nonlinear mappings between layers, offers superior performance for intricate scenarios such as stock market predictions (Gupta and Katarya, 2021). Ni et al. combined the strengths of recurrent neural networks (RNN) and convolutional neural networks (CNN) to introduce the C-RNN forecasting approach for forex time-series data (Ni et al., 2019); improving prediction accuracy through deep-learning algorithms. In 2017, Vaswani et al. introduced the Transformer model, employing a novel architecture that leverages attention mechanisms to overcome limitations of conventional CNN and RNN models (Vaswani et al., 2017). This model has shown exceptional effective in natural language processing (NLP) tasks by enabling parallel training and capturing global context through its self-attention mechanism. However, its application has been mostly limited to sentiment analysis (Köksal and Özgür, 2021); with few studies exploring its potential in stock market forecasting. Wang et al. conducted back-testing experiments on major global stock indices, such as CSI 300, S&P 500, Hang Seng Index, and Nikkei 225 (Wang et al., 2022), demonstrating that the Transformer model, with its encoder-decoder structure and multi-head attention mechanism, significantly outperforms existing methods in identifying

^{*} Corresponding author.

E-mail addresses: 64612007@kmitl.ac.th (L. Feng), ananta.sin@kmitl.ac.th (A. Sinchai).

<https://doi.org/10.1016/j.jksuci.2024.102000>

Received 9 December 2023; Received in revised form 27 February 2024; Accepted 27 February 2024

Available online 1 March 2024

1319-1578/© 2024 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Table 1
Abbreviations and their descriptions.

Abbreviation	Description
AA	Autocorrelation attention
BERT	Bi-directional encoder representations for transformers
BiLSTM	Bi-directional long short-term memory
BiRNN	Bi-directional RNN
CNN	Convolutional neural network
CNN-LSTM	Convolutional neural network and long short-term memory
C-RNN	CNN combined with RNN
EMD	Empirical mode decomposition
FF	Feed forward
FDGRU-Trans	Frequency decomposition gate recurrent unit transformer
GAT	Graph attention network
GNN	Graph neural network
GRU	Gated recurrent unit
HGNN	Hierarchical graph neural network
IC	Influence controlling
LSTM	Long short-term memory
MAE	Mean absolute error
MAN-SF	Multipronged attention network for stock forecasting
MHA	Multi-head attention
MLP	Multi-layer perceptron
MSE	Mean squared error
NLP	Natural language processing
NMT	Neural machine translation
R ²	R-square
RAT	Rotary-positional encoding autocorrelation transformer
RNN	Recurrent neural network
SSPM	Structured stock prediction model
STLAT	Selective transfer learning with adversarial training
SVM	Support vector machine
TCN	Temporary revolution network
TEANet	Transformer encoder-based attention network
TL	Transfer learning

intrinsic stock market patterns. Despite its success, the standard Transformer model, originally designed for NLP tasks, encounters issues in financial predicting due to its point-wise self-attention mechanism's inability to capture local context and crucial cyclical and seasonal patterns in financial time series. Addressing the challenge of effectively extracting cyclical features to boost forecasting accuracy remains an immediate problem.

Transfer learning (TL) enhances the adaptability of machine learning models by leveraging knowledge from a specific source domain to enhance performance in a different target domain (Goodfellow et al., 2016). He et al. developed a unique method for predicting stock price movements utilizing TL (He et al., 2019), aiming to increase the accuracy of stock price predictions. Their approach involved using knowledge learned from one stock to predict the movements of another. Their findings indicated that TL is more effective when the source and target datasets originate from the same industry, highlighting the challenge of applying TL across different industries, regions, and types of cash instruments. These differences are due to distinct market policies, industry performance, macroeconomic data, and investor sentiment, which create unique data distributions. Transferring knowledge from a source dataset to a target dataset, which might exhibit different features or varied data distributions, remains a significant challenge to be addressed in TL.

This study introduces a novel model, the rotary-positional encoding autocorrelation Transformer (RAT), designed to solve such problems by integrating TL across various markets with a modified Transformer model. The RAT model combines rotary-positional encoding with autocorrelation to specifically tailor it for forecasting financial market prices. First, a standard Transformer model is adapted by incorporating rotary-positional encoding and replacing its self-attention mechanism with autocorrelation, rooted in the innate periodicity of time series. This shift enables the model to identify dependencies and aggregate representations at the subseries level, enhancing its capability to capture cyclical features in financial time series. Traditional positional encoding

offers absolute positional information, which may not fully capture long-term dependencies. In contrast, rotary-positional encoding within the RAT model offers a nuanced approach by encoding positions through rotation matrices, embedding explicit relative positional dependencies alongside autocorrelated representations. This innovative combination equips the RAT model with an enriched understanding of temporal relationships, significantly improving its capability to capture and model the intricacies of financial data points.

Given the global prominence and dynamic nature of the U.S. stock market, it was selected as the source domain for this study. The research began by pre-training the RAT model on the U.S. stock market, specifically using the Dow Jones Index. Subsequently, through TL, this pre-trained model was applied to various other stock indices, commodity markets, and cryptocurrencies. To ensure improved forecasting accuracy, fine-tuning was done on the target datasets. The study's significant contributions to advancing TL in financial market forecasting include:

1. Introducing a TL model using the RAT model for multi-day predictions across different cash instruments,
2. Employing rotary-positional encoding to enhance prediction accuracy over extended sequences,
3. Implementing an autocorrelation technique to extract and leverage cyclical feature similarities,
4. Conducting a comprehensive performance evaluation of multi-day predictions across 12 different cash-instrument datasets to illustrate the efficacy of the proposed TL model.

The structure of this paper is structured as follows: Section 2 offers a review of relevant literature. Section 3 describes the foundational theories related to this research. The architecture of the proposed model is thoroughly detailed in Section 4. The experimental methodology and a discussion of the results are presented in Sections 5 and 6, respectively. Lastly, Section 7 concludes the study, summarizing its findings and contributions.

2. Literature review

Deep learning has seen widespread application across various domains in recent years (D'Angelo et al., 2021; Ying et al., 2021), with time-series prediction becoming increasingly dominated by models based on artificial neural networks, such as RNNs, CNNs, and LSTMs. RNNs often struggle with the vanishing gradient problem when processing data over extended periods. To address this, variants such as LSTM and gated recurrent units (GRUs) have been developed. LSTMs, celebrated for their ability to remember information over long periods, excel in processing textual data and predicting stock market trends (Hochreiter and Schmidhuber, 1997). In the field of stock market prediction, Feng et al. introduced an enhanced predictive model by integrating attentive LSTM with adversarial training (Feng et al., 2018), while Chen et al. devised a BiLSTM model to encode stock data alongside financial news within their SSPM and multi-task SSPM frameworks (Chen et al., 2019). GRUs, evolving from LSTMs, offer enhanced training efficiency in stock market predictions (Chung et al., 2014), as demonstrated by Li and Pan, who explored the lasting or transient effects of news on stock prices through psychological and economic lenses (Li and Pan,), creating a hybrid deep-learning model that combines LSTM and GRU components. Yang et al. introduced a GRU-based model featuring a dual-level attention mechanism, which applies input attention to weight financial news headlines according to their influence on stock prices (Yang et al., 2018). Despite LSTMs and GRUs being more adept at capturing long-term dependencies than traditional RNNs, they have limitations in memory capacity and may not effectively capture historical information from the distant past. The fusion of CNNs with LSTMs significantly enhances time-series prediction by combining the feature extraction capabilities of CNNs with the predictive prowess of LSTMs. For instance, Lu et al. proposed a CNN-LSTM model designed to predict

daily stock closing prices by analyzing features from a 10-day historical series, with the LSTM generating future price predictions (Lu et al., 2020). Lu et al. also devised a CNN-BiLSTM-AM model that exploits an attention mechanism to identify significant historical price movements, thereby improving the performance of CNN-based forecasts (Lu et al., 2021). While CNNs are proficient at identifying short-term and local patterns, they might overlook key information such as long-term trends and cycles, which are crucial for accurately forecasting financial time series.

Graph neural networks (GNNs) are a specialized category of artificial neural networks optimized for processing graph-structured data and have seen considerable application in stock market prediction tasks (Scarselli et al., 2008). An example is the HGNN model by Xu et al. (Xu et al., 2022), which innovatively predicts stock price limits by analyzing market-state attributes and constructing relational graphs for stock markets. Modeling has been further enhanced by graph attention networks (GATs) which amalgamate attention layers to enhance performance on large-scale graphs by focusing on pivotal nodes. An example is the MAN-SF network by Sawhney et al. (Sawhney et al., 2020), which synergistically combines financial data, social media insights, and inter-stock relationships. However, GNNs, mainly designed to analyze spatial relationships, such as connections between companies, face challenges in dealing with time-varying financial time-series data. Additionally, GNNs are more complex and typically require longer training times compared to traditional neural networks models.

The Transformer model has emerged as a powerful tool for modeling long-range dependencies, benefiting from its ability to parallelize computations, and has outperformed various models across numerous NLP tasks. Its adaptation for financial time-series prediction has led to significant performance improvements. Zhang et al. proposed TEANet, which leverages both Twitter data and stock prices for accurate stock movement predictions (Zhang et al., 2022), utilizing the Transformer's multiple attention mechanisms for effective feature extraction. Wang et al. demonstrated the Transformer's superiority in predicting stock market indices (Wang et al., 2022) through back-testing on major global indices such as CSI 300, S&P 500, Hang Seng Index, and Nikkei 225, showcasing its ability generate superior investor returns. The Informer, an advanced Transformer variant, was evaluated by Lu et al. against LSTM, Transformer, and BERT models over various time intervals and stocks, and market indices (Lu et al., 2023), showing remarkable improvements in prediction accuracy. Another innovative model, Bi-LSTM-TRAN-TCN by Wang, combines Bi-LSTM, Transformer, and TCN (Wang, 2023) to process bi-directional sequence data, capture long-range dependencies, and recognize sequence patterns, respectively. Additionally, Li et al.'s FDGRU-trans model for stock price prediction (Li and Qian, 2022) employs EMD to split stock data into trends and modes, enhancing feature extraction from complex signals by integrating GRU, LSTM, and Transformers. Despite these advancements, Transformer derivatives using sparse self-attention for efficiency in processing long

sequences may encounter information bottlenecks, potentially limiting their ability to fully capture periodic features and trends in financial time series.

TL enables models to leverage knowledge acquired from one domain and apply it to new domains (Pan and Yang, 2009). This approach facilitates efficient adaptation to new tasks, shortens training durations, and improves generalization capabilities. Additionally, it addresses the problem of insufficient training data often encountered in deep learning. He et al. introduced a novel TL methodology that leverages two source datasets to enhance the prediction accuracy of a target dataset (Goodfellow et al., 2016). Their experimental findings underscore the advantage of using a dual-dataset TL framework over conventional methods. Wu et al. tackled stock price prediction by integrating TL with deep-learning models, informed by industrial chain information (Wu et al., 2022). Their study covered trends across 379 indices in the Chinese stock market, contemplating the interactions between upstream and downstream enterprises, and found that the MLP method outperformed other models in accuracy when incorporating industrial chain information. Li et al. introduced an advanced stock movement prediction model, STLAT (Li et al., 2022), which elevates prediction accuracy through pre-training and fine-tuning methods, including a data selector module. The shared-base model underwent pre-training with tasks related to prediction, adversarial training, and data selection. The efficacy of the model was validated through testing on three distinct datasets. Merello et al. proposed a market-return regression approach to forecast stock market trends (Merello et al., 2019); overcoming traditional forecasting methods' limitations by employing TL and assigning varying weights to data samples based on their significance. This method yielded higher profits with fewer trades in real trading simulations. However, most current methods focus on single stock markets or specific industries, neglecting the potential benefits of exploring market interactions to enhance financial time-series prediction accuracy.

The literature review categorizes and summarizes studies in financial price prediction Table 2, presenting a range of models alongside their advantages and weaknesses.

To counter the shortcomings shown in Table 2, this study proposes a TL model that incorporates a RAT. This model efficiently extracts both periodic and trend features from financial time series and boosts prediction accuracy by exploiting interactions between markets.

3. Background

This section outlines the preliminary concepts underlying the development of a TL model tailored for predicting cash-instrument prices. The TL model presented in this study merges the principles of the Transformer model with autocorrelation to enhance feature capture in time-series data, particularly at the subseries level.

Table 2
Summary of financial price forecasting techniques.

Model	Algorithm	Advantage(s)	Weakness(es)
RNN-based	LSTM (Feng et al., 2018)(Chen et al., 2019); GRU (Li and Pan,)(Yang et al., 2018)	1. Capturing long-term dependencies	1. Limited memory 2. Ineffective capturing historical information from the distant past
CNN-based	CNN-LSTM (Lu et al., 2020) CNN-BiLSTM-AM (Lu et al., 2021)	1. Capturing short-term and local patterns	1. Overlooking important information such as long-term trends and cycles
GNN-based	HGNN (Xu et al., 2022); MAN-SF (Sawhney et al., 2020)	1. Effective yielding the correlations and dependencies among various assets in the financial market	1. Computational complexity 2. Long training times
Transformer-based	Transformer (Lu et al., 2023); TANet (Zhang et al., 2022); Informer (Wang, 2023); BiLSTM-MTRAN-TCN (Li and Qian, 2022), FDGRU-Trans (Pan and Yang, 2009)	1. Capturing long-term dependencies 2. Enabling parallel computation of self-attention mechanism	1. Ineffective identifying period-based dependencies through point-wise self-attention
Transfer learning		1. Applying prior knowledge learned from one domain to new domains	1. Lacking exploration into cross-market interactions

3.1. Transformer model

The development of the Transformer model was pivotal, focusing on integrating the attention mechanism to improve the processing of sequential data (Vaswani et al., 2017). Since their emergence, various models designed for a broad range of tasks across different disciplines have been introduced, aiming to streamline the interpretation and processing of sequences. Transformer variants have shown outstanding results, setting new benchmarks in NLP translation and becoming widely used. While some models preserve the original Transformer structure, others may employ only its encoder or decoder components. The performance of Transformer-based models and their suitability for specific tasks very greatly, according to the architectural choices made. However, the self-attention mechanism remains a core feature across these models, with most employing either self-attention or variants of MHA mechanism as their principal learning layers. Understanding the self-attention mechanism is essential due to its critical role in the functionality of Transformer.

3.1.1. Attention mechanism

The concept of applying attention mechanisms within neural networks can be tracked back to the late 1980 s, with one of the earliest implementations aimed at enhancing the neocognitron (Fukushima, 1980) with selective attention capabilities (Fukushima, 1987). This approach was further adapted for tasks such as segmenting connected characters in cursive handwriting (Fukushima and Imagawa, 1993). Subsequent improvements in attention mechanisms spanned various applications, including detecting targets (Schmidhuber and Huber, 1991). These early studies laid the groundwork for further investigation within neural networks. By the 2000 s, research efforts continued to evolve, introducing models that combined attention direction with object recognition (Miau and Itti, 2001) and developing computational models to simulate human eye movements for object class (Zhang et al., 2005) detection and visual selective attention, aiming to identify relevant areas within visual stimuli (Le Meur et al., 2006). In 2010, a pioneering system inspired by the human retina's fovea was introduced for image classification (Larochelle and Hinton, 2010), combining training with a restricted Boltzmann machine and an attention-driven fixation controller. The prominence of attention mechanisms surged in 2015 with the introduction of a new NMT method (Bahdanau et al., 2015),

enhancing NMT models with a BiRNN encoder for improved semantic understanding. This period marked the start of attention mechanisms playing a crucial role in NLP and deep learning (Soydaner, 2022). In 2017, the introduction of the Transformer model, a new neural network architecture focusing on self-attention, represented a significant advancement (Vaswani et al., 2017), achieving outstanding results in machine translation and English constituency parsing. This architecture's distinctive absence of recurrence and convolution, relying instead on self-attention, has been instrumental in advancing the understanding of attention mechanisms, enabling better recognition of local features and reducing dependence on external information to improve model performance.

The Transformer model uses the "Scaled Dot-Product Attention" mechanism, which involves three key matrices: Query (Q), Key (K), and value (V). Each of these matrices encodes specific details of each input in the sequence. The attention mechanism calculates its output using the Softmax function, assigning a probability score to the weighted combination of these matrices, as shown in Fig. 1. The scaled dot-product attention function, defined as $\text{Attention}(Q, K, V)$, is formulated in Eq (1). In this equation, matrices Q and K correspond to the Query and Key vectors, respectively, while matrix V stands for the value vectors. Each matrix has a dimension of d_k .

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

3.1.2. Multi-Head attention

MHA comprises multiple "heads," with each head possessing its own set of queries, keys, and values, generated by fully connected layers from the original input data, as expressed in Eq (2). In this equation, head_i represents the attention mechanism at the i head, with W_i^Q , W_i^K , and W_i^V representing the learnable parameter matrices for that head. The advantage of MHA is its ability to capture different contextual information from each head, combining them into a complicated output.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2)$$

The outputs from each head MHA are merged into a single output layer through a concatenation function, *Concat*, and then multiplied by a weight matrix W^O , as represented in Eq (3).

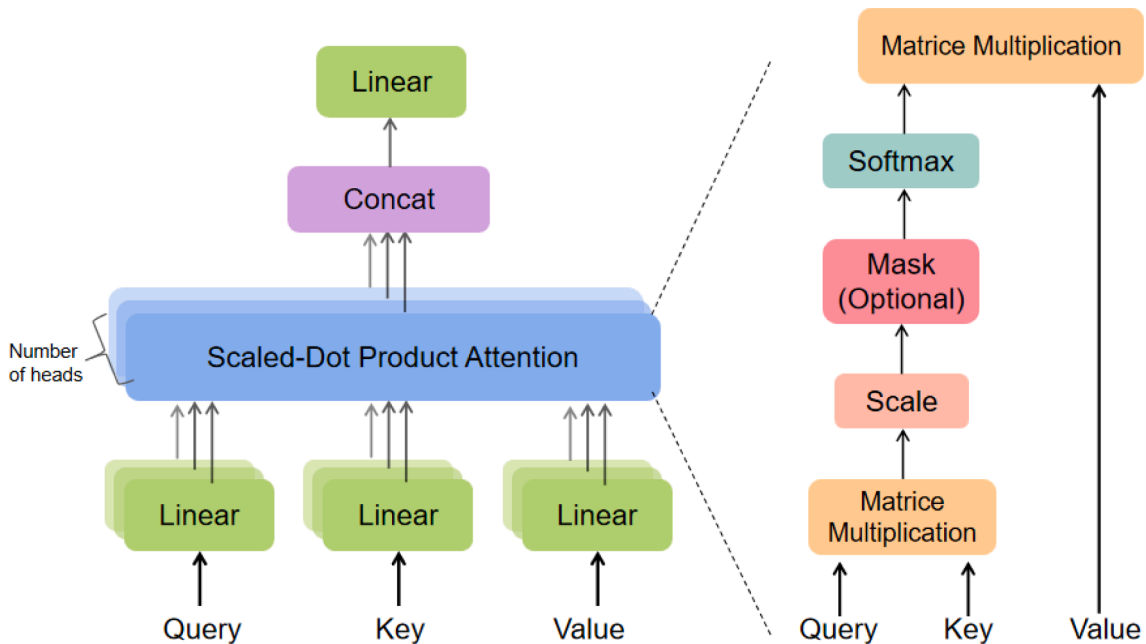


Fig. 1. Structure of MHA (left) and process of scaled dot-product attention (right).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_i) W^o \quad (3)$$

This weight matrix W^o integrates the diverse features from all heads. The result of this operation is the MHA function, defined as $\text{MultiHead}(Q, K, V)$.

MHA improves the representation of input contexts by combining different aspects of the attention mechanism, whether focusing on proximal or distal information. This method enables the model to aggregate attention from different perspectives, leading to improved performance. However, the original Transformer model comprises not only the MHA component but also includes an encoder and a decoder, each containing layers that employ an attention mechanism. Specifically, the attention mechanism is executed in parallel multiple times within the Transformer architecture, illustrating the role of MHA. The encoder and decoder are detailed sequentially below.

3.1.3. Encoder

Typically, the Transformer architecture features one principal encoder block composed of several sub-encoders. The encoder transforms an input sequence into a sequence encoded with features. Each sub-encoder includes three main components: the MHA layer, the add and norm layers, and FF layer, as depicted in Fig. 2. For simplicity, it is assumed that there is only one head attention.

The encoder operates as follows: After the input sequence vector, x , is embedded and undergoes positional encoding to produce the encoded input, x_{en} . Then, x_{en} is projected into the vectors of Q, K, V through the dot-product with each corresponding weight of Q, K, V , resulting in Eqs. (4)–(6).

$$Q = W^Q x_{en} \quad (4)$$

$$K = W^K x_{en} \quad (5)$$

$$V = W^V x_{en} \quad (6)$$

Subsequently, the vectors of Q, K, V are fed into the MHA layer and then proceed to the add and norm layers. In the add and norm layers, the output vectors produced by the MHA layer are normalized as per Eq (7), where the *Output* variable represents the normalized *Input*, and the *Input* variable is initial input vector. The operator $E[\cdot]$ denotes the expectation of the *Input*, σ^2 is the variance of the *Input*, and ε provides numerical stability. The last two variables, γ and β , are learnable parameters. Once the output vectors of Q, K , and V are normalized, they are added to the original vectors that passed through the MHA layer.

$$\text{Output} = \left(\frac{\text{Input} - E[\text{Input}]}{\sqrt{\sigma^2 + \varepsilon}} \right) \cdot \gamma + \beta \quad (7)$$

Following the MHA and add and norm layers, the updated vectors for Q, K , and V are acquired. These vectors are then sequentially passed through the FF layer and the add and norm layers. The FF layer executes feed-forward neural network calculations, with the output being normalized again using Eq (7), resulting in another set of normalized output vectors. These vectors are then combined with the updated vectors of Q, K , and V to produce the feature-encoded vectors for each.

This process outlines the operation of a single encoder within the Transformer architecture. If the structure includes many encoders stacked sequentially, the encoding process proceeds from the first (bottom) encoder to the last (top) encoder. Once the feature-encoded vectors of Q, K , and V are obtained, only the vectors for K and V are transmitted to the decoder.

3.1.4. Decoder

The decoder in the Transformer architecture mirrors the structure of the encoder but introduces an additional masked MHA layer at the outset. This layer employs scaled dot-product attention with masking to exclude future tokens from the prediction, focusing only on preceding outputs. The decoder utilizes the attention mechanism in two instances: first, to process attention utilizing vectors of Q, K , and V computed according to Eqs. (4)–(6); second, to apply attention with vectors of K and V received from the encoder, facilitating the transfer of knowledge from the source to the target domain. Each processed attention vector is then refined in the feed-forward unit, enhancing clarity for downstream layers. The output from the decoding process is further processed by linear (fully connected) and Softmax layers at the top of the decoder, generating the final output of the Transformer architecture. This iterative cycle continues until the processing of the last token is completed, as depicted in Fig. 2.

3.1.5. Positional encoding

Positional encoding is a technique used to maintain the order of input sequences after embedding, compensating for the Transformer model's lack of recurrence and convolution. This process assigns a unique value to each position in the input sequence, ensuring that the model can recognize the sequence's order. In the original Transformer model, absolute positional encoding is implemented, which maps the input sequence's position to a value range between 0 and 1 using trigonometric functions, as outlined in Eqs. (8)–(9).

$$PE(pos, 2k) = \sin\left(\frac{pos}{10000^{\left(\frac{2k}{d}\right)}}\right) \quad (8)$$

$$PE(pos, 2k+1) = \cos\left(\frac{pos}{10000^{\left(\frac{2k}{d}\right)}}\right) \quad (9)$$

In these equations, pos represents the position of an element within the input sequence, where $0 \leq pos < n/2$ and n -variable is the length of the input. The dimension of the input sequence is denoted by d , and k is the index for positional mapping, ranging from zero to $d/2$.

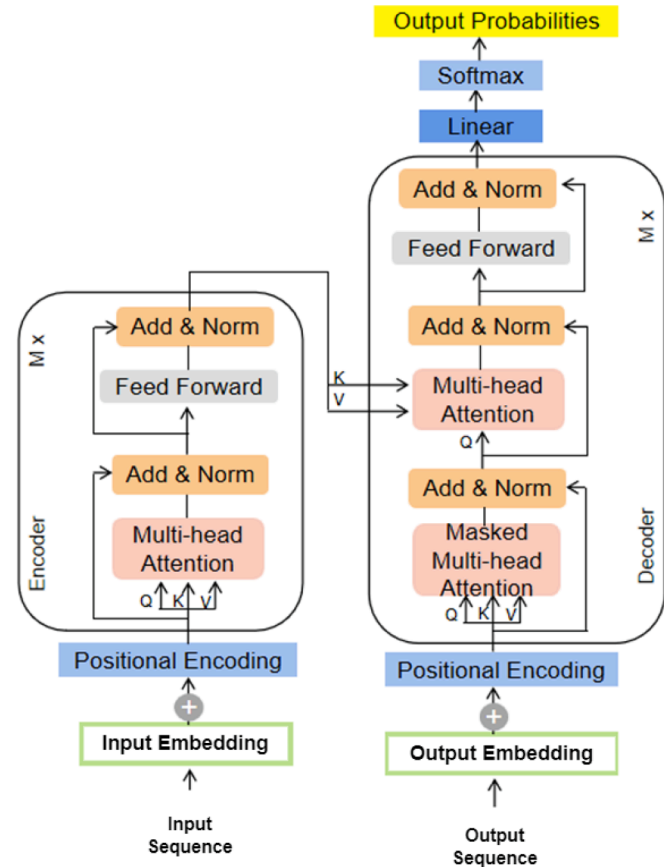


Fig. 2. Transformer architecture has encoder (left) and decoder (right).

3.1.6. Input embedding

Input embedding transforms an input word into a numeric vector using byte-pair encoding. However, this study focuses on numerical time-series data from cash-instrument markets, where the byte-pair encoding technique is not applicable. Instead, input embedding is achieved through general one-dimensional (1D) CNNs.

3.2. Autocorrelation

Autocorrelation measures the similarity between a time-series and its lagged versions over successive time intervals, highlighting the relationship between a variable's present and past values (Dondurur, 2018). This mathematical operation is widely used in time-series analysis to assess how similar a time signal is to itself across different periods. The autocorrelation of a discrete-time series, denoted as $S_{xx}(\delta)$, is calculated using Eq (10) by summing the product of $x[t]$ and $x[t-\delta]$, where $x[t]$ is the value of the time series at time t , and $x[t-\delta]$ is the value at a lag of δ .

$$S_{xx}(\delta) = \frac{1}{N} \sum_{t=0}^{N-1} x[t]x[t-\delta], \delta \geq 0 \quad (10)$$

The subscript 'xx' represents the correlation sequences, and δ represents the lag factor, or time-shift parameter, in the autocorrelation process. For a time series comprising N data points, it is noteworthy that the resulting autocorrelation sequence, also known as an autocorrelation, contains $2N-1$ data points.

3.3. Transfer learning

Conventional machine learning operates under the assumption that the training and testing data possess identical input features and distributions. However, discrepancies in these datasets distributions can significantly degrade the performance of predictive models (Weiss et al., 2016)(Shimodaira, 2000). Often, gathering training data that perfectly matches the feature and distribution properties of the testing data is both challenging and costly. This scenario underscores the importance of devising efficient models for specific target domains using data from a related source domain, leading to the exploration of TL. TL aims to apply knowledge acquired in one domain to enhance learning in another domain and is emerging as a promising approach in machine learning. The concept of TL has roots in educational psychology, with the generalization theory of transfer by psychologist C. H. Judd (Judd, 1908) explaining that learning can be transferred through the generalization of past experiences. Transfer is considered successful when an individual can apply these experiences from one context to another, contingent upon the existence of a link between the two learning activities. Because of its impressive performance, TL has found widespread application in various fields, including time-series forecasting. In (Laptev et al., 2018); the authors investigated TL's application in time-series forecasting by designing a reconstruction loss function that seamlessly incorporates TL into the model's architecture. In (Gupta et al., 2018); a novel TL framework using deep RNNs is developed for clinical time-series classification, structured around two phases; initial training of a pre-trained RNN model on multiple source datasets, followed by a logistic regression model that employs target features extracted by the RNN model. TL involves three key concepts: domain, task, and the TL process itself, with definitions provided in Table 3.

Of that is worth noting that similarities between domains do not guarantee easier learning; in fact, such similarities can sometimes lead to confusion. Psychologically, this phenomenon is known as negative transfer, where past experiences hinder the learning of new tasks. Similarly, in TL, negative transfer refers to situations where transferred knowledge adversely impacts the learning process in the target domain.

Table 3

Keywords in TL.

Keywords	Definition
Domain	A domain $\mathcal{D}$ includes two elements: a feature space $\mathcal{X}$ and a marginal probability distribution $P(X)$, where $X = \{x_1, \dots, x_n\} \in \mathcal{X}$. In TL, the domain with existing knowledge is known as the source domain, often denoted by $\mathcal{D}_s$, while the domain where new knowledge needs to be acquired is referred to as the target domain, typically symbolized by $\mathcal{D}_T$.
Task	A task comprises a label space $\mathcal{Y}$ and an objective predictive function $f(\cdot)$ (represented as $\mathcal{J} = \{\mathcal{Y}, f(\cdot)\}$). This function, learned from training data pairs $\{x_i, y_i\}$, where $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$, aims to approximate the conditional probability $P(y x)$ for predicting labels of new instances x . Practically, domains are frequently represented through various instances, some labeled and others not.
TL	For simplicity, this definition focuses on the scenario involving a single-source domain $\mathcal{D}_s$ and a single target domain $\mathcal{D}_T$, reflecting the most frequently encountered research situation. Specifically, the data from the source domain is referred to as $\mathcal{D}_s = \{(x_{s1}, y_{s1}), \dots, (x_{sn}, y_{sn})\}$ where $x_{si} \in \mathcal{X}_s$ is the data instance and $y_{si} \in \mathcal{Y}_s$ is the corresponding output. Similarly, the target domain is expressed by $\mathcal{D}_T = \{(x_{t1}, y_{t1}), \dots, (x_{tn}, y_{tn})\}$, where $x_{ti} \in \mathcal{X}_T$, $y_{ti} \in \mathcal{Y}_T$ are the corresponding outputs. TL involves using knowledge from a source domain $\mathcal{D}_s$ and task $\mathcal{J}_s$ to enhance learning in a different target domain $\mathcal{D}_T$ and task $\mathcal{J}_T$. The goal is to improve the target predictive function $f(\cdot)$ in $\mathcal{J}_T$, even when $\mathcal{D}_s \neq \mathcal{D}_T$ or $\mathcal{J}_s \neq \mathcal{J}_T$.

4. Proposed model

In this section, we propose a TL model that employs the RAT model. The original structure of the Transformer model, depicted in Fig. 2, has been modified to incorporate the RAT model, as illustrated in Fig. 3. For brevity, this TL model adopting the RAT approach will henceforth be referred to as the proposed RAT approach throughout this paper. The implementation of the proposed RAT approach, as shown in Fig. 3, necessitates the inclusion of a sequence decomposition layer and MLP layers. However, the focus is primarily on the sequence decomposition; the MLP layer, being a standard configuration of 1D CNNs, is not described upon here. The core of the proposed RAT approach lies in the integration of rotary-positional encoding and autocorrelation mechanisms.

The proposed RAT approach modifies the Transformer architecture by initially incorporating a sequence decomposition layer to disintegrate an input time-series signal into seasonal and trend or cyclical component signals. Sequence decomposition also occurs after the autocorrelation mechanism layer in both the encoder and decoder. The MLP layer replaces the original FF layer in both components. The autocorrelation mechanism substitutes for the MHA and masked MHA layers in the traditional Transformer structure. Additionally, a rotary-positional encoding layer is introduced to replace the positional encoding and input embedding layers for the vectors of Q and K , while the vector of V is still dependent on positional encoding and input embedding. An extra output sequence is added to the decoder. The functionality of each layer is explained sequentially. Lastly, the prediction layer replaces the linear and Softmax layers at the top right of the conventional Transformer model displayed in Fig. 2. The workflow of the proposed RAT approach is detailed at the end of this section.

4.1. Sequence decomposition layer

In every financial market, a multitude of factors, including economic and political influences, cause cash-instrument prices to fluctuate in complex patterns, generating a time-series signal. To initially analyze this time-series signal, denoted as x , a sequence decomposition layer is utilized. This layer divides the time-series signal into two components: a cyclical or trend component, and a seasonal component. The cyclical component, denoted as x_c , is attained by calculating the moving average of x , as represented in Eq (11).

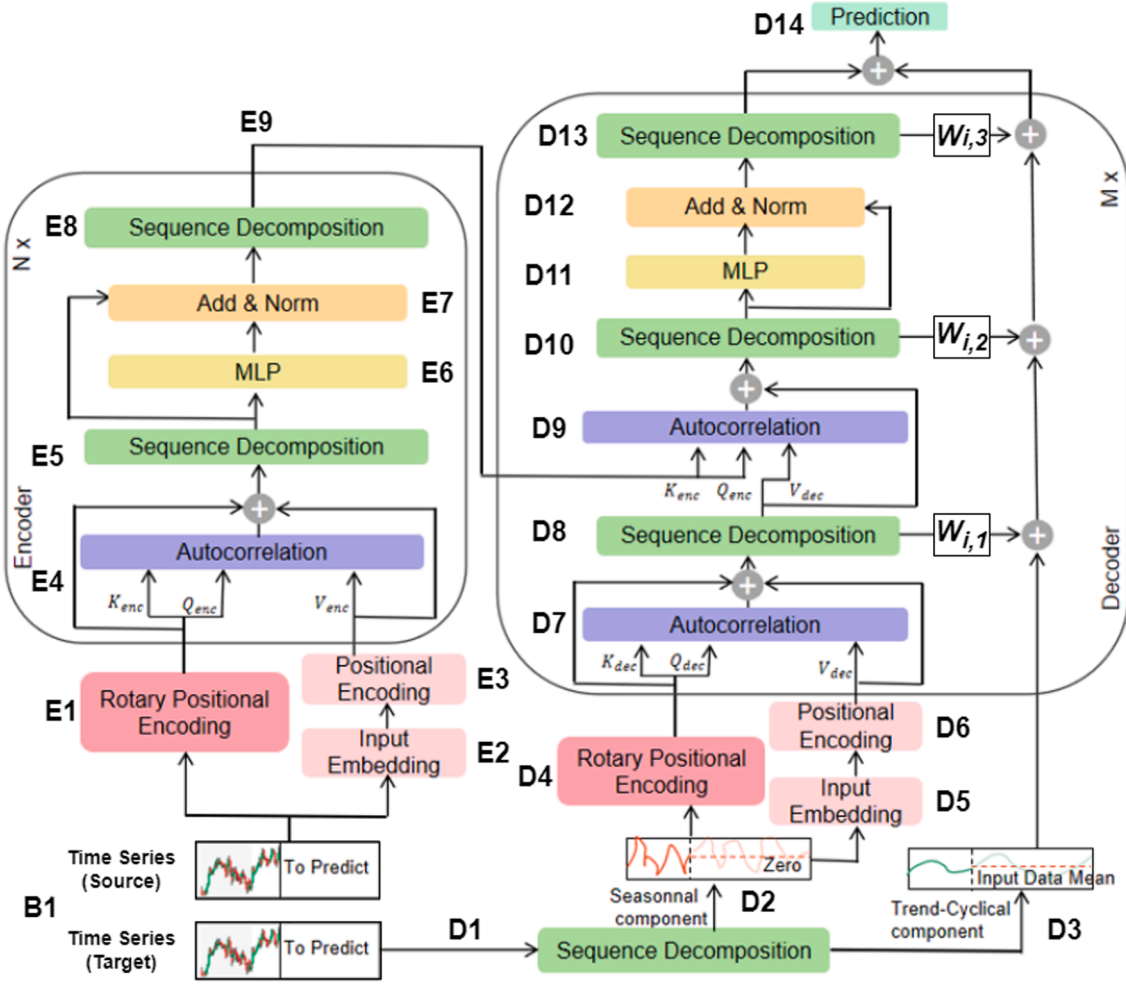


Fig. 3. Structure of TL model adopting RAT model.

$$x_c = \text{AvgPool}(\text{Padding}(x, \text{kernel size}), \text{kernel size}, \text{stride}) \quad (11)$$

In Eq (11), the time-series signal x is initially padded using the $\text{Padding}(\bullet)$ function to maintain a consistent length. This function duplicates data at both the beginning and end of x , based on the specified number of pads. After padding, the modified x undergoes a moving average calculation via the $\text{AvgPool}(\bullet)$ function, utilizing a predefined kernel size and stride. The seasonal component signal, x_c , is then derived by subtracting the cyclical component x_e from x , as expressed in Eq (12). The operations in Eqs. (11)–(12) are collectively defined as the $\text{SeqDecomp}(\bullet)$ function, with its procedural steps outlined in Algorithm 1.

$$x_s = x - x_c \quad (12)$$

Algorithm 1 $\text{SeqDecomp}(\bullet)$ function

Input: x , kernel size, stride

Output: x_c, x_s

1: number of pads $\leftarrow$ Do the floor division of (kernel size – 1) by 2

2: front pad $\leftarrow$ Duplicate the value of x at the first position with the number of pads

3: end pad $\leftarrow$ Duplicate the value of x at the last position with the number of pads

4: $x_p \leftarrow$ Concatenate (front pad, x , end pad) altogether with this manner

5: $x_c \leftarrow$ Do the moving average on x_p with the kernel size window and the stride slide

6: $x_s \leftarrow$ Subtract x_c out of x

7: **return** x_c, x_s

4.2. Rotary-Positional encoding layer

The classical Transformer model employs absolute positional encoding, which does not account for the relative positions of data points. Absolute positional encoding yields a unique positional vector for each input in the time series by employing a specific encoding vector of the same dimensionality as the input. However, this method does not capture the positional relationships across the entire time series, leaving the relations between consecutive input positions unknown. The lack of this positional information means absolute positional encoding cannot recognize patterns related to distances between points in the time series. To harness the strengths of both absolute and relative positional encodings, we propose rotary-positional encoding, which surpasses the traditional absolute encoding method in this study. Designed for simplicity in computational complexity and mathematical expression, rotary-positional encoding operates in two dimensions (2D) (Su et al., 2024). Within the Q , K , and V vectors is chosen for applying rotary-positional encodings. For the proposed RAT approach, the vectors of Q and K are selected.

Implementing rotary-positional encoding involves transforming Eqs. (4)–(5) into polar coordinate form. Instead of employing x_{en} according to Eqs (4)–(5), two values representing relative positions, x_m and x_n , where m and n denote positions, are utilized. This transformation incorporates Euler's formulas with angle arguments of $m\theta$ and $n\theta$, modifying Eqs. (4)–(5) to produce Eqs. (13)–(14), respectively.

$$Q_m = W^Q x_m e^{im\theta} \quad (13)$$

$$K_n = W^K x_n e^{jn\theta} \quad (14)$$

Eqs. (13)–(14) accommodate both absolute and relative positional encodings, with m and n representing absolute positions and θ representing relative positioning. The process of consolidating relative positional encoding effectively rotates the angle θ . To define the rotary-positional encoding function, the inner product of the transposes Q_m from Eq (13) and the conjugate of K_n from Eq (14) is calculated, denoted as function g in Eq (15), where the $*$ -superscript indicates a complex conjugate.

$$g(x_m, x_n, m - n) = Q_m^T K_n^* = W_{Q,K}^Q (W^K x_n)^* e^{j(m-n)\theta} \quad (15)$$

Focusing solely on the real component of Eq (15), the rotary-positional encoding function $f_{\{Q,K\}}$ with x_m and m as its arguments is represented in Eq (16). This equation breaks down the trigonometric terms $\cos m\theta$, $-\sin m\theta$, $\sin m\theta$, and $\cos m\theta$ from the $e^{j(m-n)\theta}$ term. Additionally, the parameters $W_{\{Q,K\}}^{(11)}$, $W_{\{Q,K\}}^{(12)}$, $W_{\{Q,K\}}^{(21)}$, and $W_{\{Q,K\}}^{(22)}$ are acquired from the inner products of $(W^Q x_m)$ and $(W^K x_m)^*$. The variables $x_m^{(1)}$ and $x_m^{(2)}$ represent the 2D coordinates of x_m .

$$f_{\{Q,K\}}(x_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{Q,K\}}^{(11)} & W_{\{Q,K\}}^{(12)} \\ W_{\{Q,K\}}^{(21)} & W_{\{Q,K\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \quad (16)$$

To extend Eq (16) from a 2D context to any x_i in a real space $\mathbb{R}^d$, where d is an even-dimensional space, d is first divided into sub-spaces. These sub-spaces are then combined using the inner product's linearity property, resulting in a generalized form of $f_{\{Q,K\}}(x_m, m)$, as expressed in Eq (17). The rotary matrix $R_{\Theta,m}^d$ is defined with Θ predefined as a set of θ_i where $\theta_i = 10000^{-2(i-1)/d}$, demonstrated by Eq (18). Here, i -parameter represents the dimensional index.

$$f_{\{Q,K\}}(x_m, m) = R_{\Theta,m}^d W_{\{Q,K\}} x_m \quad (17)$$

$$R_{\Theta,m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (18)$$

Following this generalization, the generalized rotary-positional encoding function from Eq (17), in conjunction with Eq (18), is operationalized into a practical function named $RoPE(\bullet)$ in Algorithm 2.

Algorithm 2 $RoPE(\bullet)$ function

Input: x, d
Output: Q, K
1: $Q \leftarrow$ Project x onto Q using Eq (4)
2: $K \leftarrow$ Project x onto K using Eq (5)
3: batch size, q-length, number of heads $\leftarrow$ Extract the values of batch size, length, number of heads from Q
4: k-length $\leftarrow$ Extract the value of length from K
5: d-key $\leftarrow$ Do the floor division of d by q-length
6: $m \leftarrow$ Create a 1D matrix containing float elements from 0.0 to q-length - 1
7: $\theta \leftarrow$ Create a 1D matrix containing d-key elements
8: For $j \leftarrow 1$ to d-key
9: $\theta_j \leftarrow e^{-\frac{(\ln(10000.0))}{d \cdot \text{keys}} (j-1)}$ // j is the for-loop index not a complex notation. //
10: EndFor
11: sin-pos-q $\leftarrow$ Create a 2D matrix with dimension of size of $m \times$ size of θ , do the matrix multiplication of m and θ , and perform the sine function on each element of sin-pos-q
12: cos-pos-q $\leftarrow$ Create a 2D matrix with dimension of size of $m \times$ size of θ , do the matrix multiplication of m and θ , and perform the cosine function on each element of cos-pos-q

(continued on next column)

(continued)

Algorithm 2 $RoPE(\bullet)$ function

13: sin-pos-q $\leftarrow$ Concatenate sin-pos-q and cos-pos-q
14: sin-pos-q $\leftarrow$ Duplicate sin-pos-q to contain batch size dimensions, each dimension repeats lists by number of heads, each list duplicates 1 sublist of existing elements, in each sublist repeats all elements 1 time
15: sin-pos-q $\leftarrow$ Manipulate sin-pos-q by taking all dimensions with q-length lists and all sublists, each sublist contains d-key elements
16: sin-pos-k $\leftarrow$ Repeat Step 11 to Step 15, changing q to k, to obtain sin-pos-k
17: cos-pos-q $\leftarrow$ Clone sin-pos-q
18: cos-pos-q $\leftarrow$ Move the elements from sin-pos-q starting from 1-position with stride of 2 to cos-pos-q starting from 0-position with stride of 2
19: sin-pos-q $\leftarrow$ Clone sin-pos-q
20: sin-pos-q $\leftarrow$ Move the elements from cos-pos-q starting from 0-position with stride of 2 to sin-pos-q starting from 1-position with stride of 2
21: cos-pos-k, sin-pos-k $\leftarrow$ Repeat Step 17 to Step 20, changing q to k, to acquire cos-pos-k, sin-pos-k
22: $Q \leftarrow$ Perform the matrix multiplication of Q and cos-pos-q
23: $\tilde{q} \leftarrow$ Concatenate $-Q$ starting from 1-position with stride of 2 and Q starting from 0-position with stride of 2
24: $\tilde{q} \leftarrow$ Perform the matrix multiplication of $\tilde{q}$ and sin-pos-q
25: $Q \leftarrow$ Sum Q with $\tilde{q}$
26: $K \leftarrow$ Repeat Step 22 to Step 25, changing q to k, to obtain K
27: **return** Q, K

4.3. Autocorrelation mechanism layer

The autocorrelation mechanism is utilized to form connections across the entire subseries, in contrast to self-attention, which concentrates on identifying dependencies at individual time points within a series. This mechanism identifies patterns of periodic dependencies by calculating sequence autocorrelation. Applying autocorrelation values as weights allows the mechanism to effectively cluster and integrate subseries with similar characteristics (Wu et al., 2021). It comprises of two essential steps: a period-interrelationship mechanism and a feature aggregation mechanism. These steps are incorporated into the Transformer's original attention mechanism within the autocorrelation layer; as depicted in Fig. 4, with further details provided on the right side of Fig. 4.

The period-interrelationship mechanism, essentially an autocorrelation process, aims to identify similar sub-processes by estimating their sequence periods. The similarity between sub-sequences is gauged by their autocorrelation coefficients, with a higher coefficient indicating greater similarity. To derive this mechanism for either a single head or multiple heads, the vectors of Q and K are predefined, meaning all encoding processes, including linear processes beneath the autocorrelation layer shown in Fig. 3, are performed and encapsulated in the vectors of Q and K . These vectors are then utilized for computing autocorrelation as per Eq (10). The Q vector represents the current input at discrete-time t , denoted as $Q[t]$, while the K vector is the complex conjugate of the lagged input $K[t]$ delayed by δ , symbolized as $K^*[t - \delta]$. Equation (10) is then adapted for the Q and K vectors, as illustrated in Eq (19). Given that Q and K vectors are generated by the same method from the same time series, the expression in Eq (19) remains an autocorrelation calculation.

$$S_{QK}(\delta) = \frac{1}{N} \sum_{t=0}^{N-1} Q[t] K^*[t - \delta], \delta \geq 0 \quad (19)$$

After processing the Q and K vectors through autocorrelation, the vector V is also adjusted to fulfill the autocorrelation mechanism. The V vector is rotated clockwise for each time-lag δ , with this rotation function named the $Whirling(\bullet)$ function of δ . Applying different time delays, $\delta_1, \delta_2, \dots, \delta_n$ to the $Whirling(\bullet)$ function results in variously shifted V vectors, as demonstrated in Fig. 5 where N is the data length. Here, n -variable pertains to the number of time delays. As depicted in Fig. 5, the $Whirling(\bullet)$ function rolls the original signal from the front (red box)

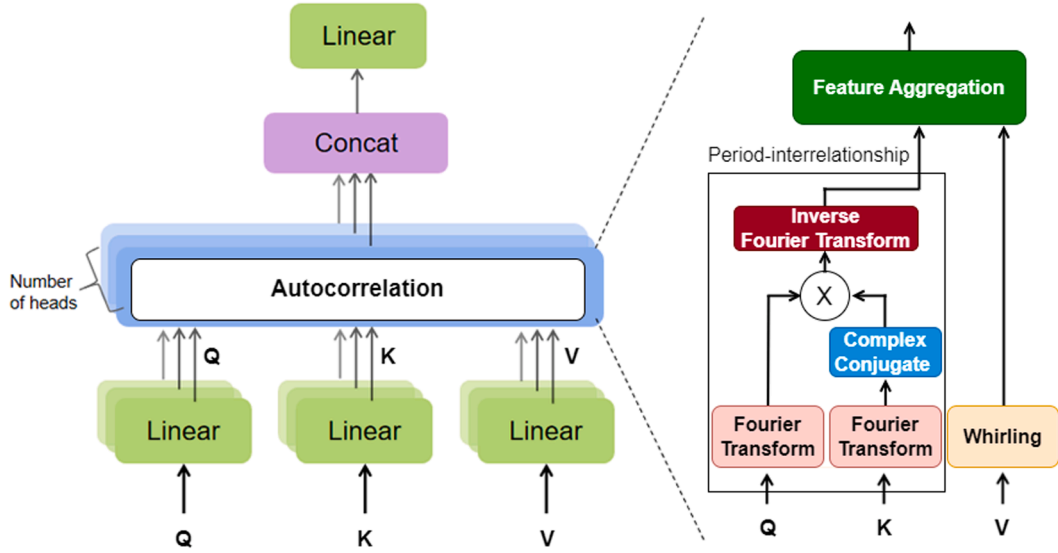
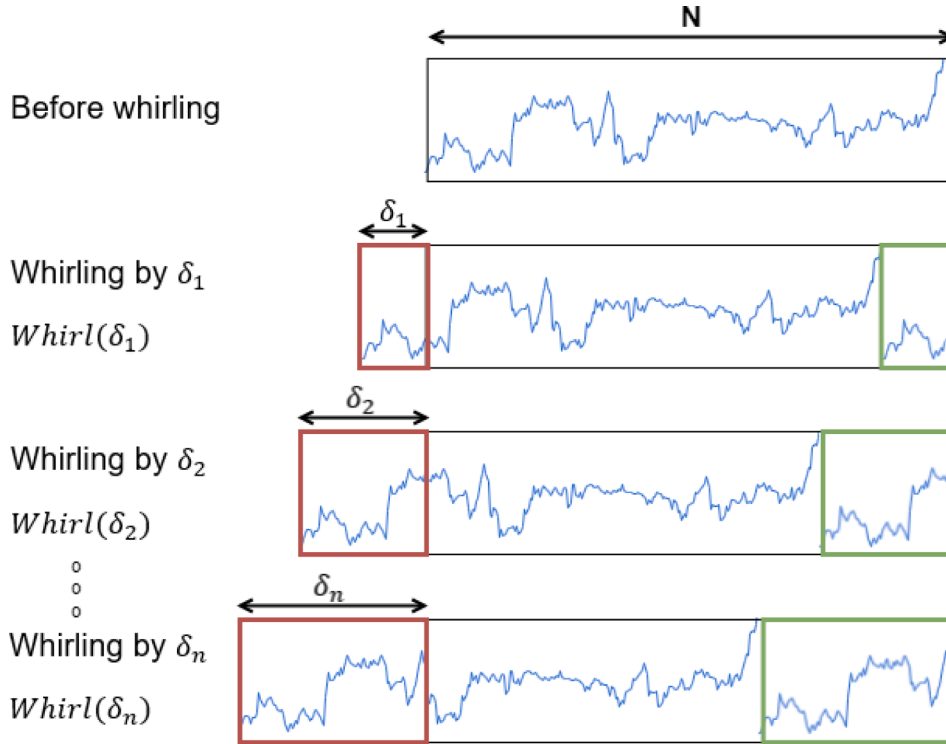


Fig. 4. Autocorrelation attention (left) with its mechanism (right).

Fig. 5. Whirling function with different time delays (topmost tray: an original signal, top tray, whirled by δ_1 , bottom tray: whirled by δ_2 and bottommost tray: whirled by δ_n).

to the back (green box) by the length of each time delay. Nonetheless, not all time delays are implemented; only significant features are selected through a feature aggregation mechanism. The operation of the *Whirling*($\bullet$) function is detailed in Algorithm 3.

Algorithm 3 *Whirling*($\bullet$) function

Input: V, δ
Output: $Roll_V$
 1: $Roll_V \leftarrow$ Shift V by the time delay length δ in a clockwise manner from front to back
 2: **return** $Roll_V$

In the feature aggregation mechanism, autocorrelation values for n -values time lags are initially computed using Eq (19), producing $S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n)$. Concurrently, the V vector is subjected to n -values time lags through the *Whirling*($\bullet$) function, yielding $Roll_V(\delta_1), Roll_V(\delta_2), \dots, Roll_V(\delta_n)$. These autocorrelation values are then sorted in descending order, facilitated by the function $\text{argTop-}n(\bullet)$, generating a new sequence of indices $\tau_1, \tau_2, \dots, \tau_n$, where τ_1 corresponds to the index of the maximum autocorrelation value and τ_n to the minimum. The reordered autocorrelation values, $R_{QK}(\tau_1), R_{QK}(\tau_2), \dots, R_{QK}(\tau_n)$, are expressed in Eqs. (21)-(22). Subsequently, these values are fed into a

Softmax function, yielding $\hat{R}_{QK}(\tau_1), \hat{R}_{QK}(\tau_2), \dots, \hat{R}_{QK}(\tau_n)$, as shown in Eq (23). Similarly, the whirled vectors are rearranged according to the autocorrelation values' descending order.

$$\tau_1, \tau_2, \dots, \tau_n = \text{argTop-n}(S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n)) \quad (20)$$

$$R_{QK}(\tau_1) = \max(S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n)) \quad (21)$$

$$R_{QK}(\tau_n) = \min(S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n)) \quad (22)$$

$$\hat{R}_{QK}(\tau_1), \hat{R}_{QK}(\tau_2), \dots, \hat{R}_{QK}(\tau_n) = \text{Softmax}(R_{QK}(\tau_1), R_{QK}(\tau_2), \dots, R_{QK}(\tau_n)) \quad (23)$$

After sorting and applying the Softmax function, the number of p -features is calculated using Eq (24), where p is the result of the floor multiplication of the hyper-parameter c and the logarithm of N , the data length, to calculate autocorrelation attention.

$$p = \lfloor c \times \log N \rfloor \quad (24)$$

This process involves summing the products of the first p numbers of $\text{Roll}_V(\tau_1), \text{Roll}_V(\tau_2), \dots, \text{Roll}_V(\tau_p)$ and $\hat{R}_{QK}(\tau_1), \hat{R}_{QK}(\tau_2), \dots, \hat{R}_{QK}(\tau_p)$, as detailed in Eq (25). For simplicity, the *Autocorrelation-Attention*($\bullet$) function is abbreviated to the *Autocorrelation*($\bullet$) function, as outlined in Eq (26). This function applied to a single head, but for multi-head configurations, the autocorrelation function is performed according to Eqs. (2)–(3).

$$\text{Autocorrelation-Attention}(Q, K, V) = \sum_{i=1}^p \text{Roll}_V(\tau_i) \bullet \hat{R}_{QK}(\tau_i) \quad (25)$$

$$\text{Autocorrelation}(Q, K, V) = \sum_{i=1}^p \text{Roll}_V(\tau_i) \bullet \hat{R}_{QK}(\tau_i) \quad (26)$$

The mechanisms of the period-interrelationship and feature aggregation are practically implemented in Algorithms 4 and 5, respectively.

Algorithm 4 Period-interrelationship mechanism

Input: Q, K
Output: S_{QK}
1: $Q \leftarrow$ Calculate the Fourier Transform using the Fast Fourier Transform method on Q
2: $K \leftarrow$ Calculate the Fourier Transform using the Fast Fourier Transform method on K
3: $K^* \leftarrow$ Perform the complex conjugate of K
4: $S_{QK} \leftarrow$ Do the Inverse Fast Fourier Transform method on the product of Q and K^*
5: **return** S_{QK}

Algorithm 5 Mechanism of feature aggregation

Input: $Q, K, V, c, N, \delta_1, \delta_2, \dots, \delta_n$
Output: $\hat{A}$
1: For $j \leftarrow 1$ to n
2: $S_{QK}(\delta_j) \leftarrow$ Compute Eq (19) using the argument of δ_j as input by Algorithm 4
3: $\text{Roll}_V(\delta_j) \leftarrow$ Plug δ_j into *Whirling*($\bullet$) run by Algorithm 3
4: EndFor
5: $\tau_1, \tau_2, \dots, \tau_n \leftarrow$ Sort and locate by $\text{argTop-n}(S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n))$
6: $R_{QK}(\tau_1), R_{QK}(\tau_2), \dots, R_{QK}(\tau_n) \leftarrow$ Reorder $S_{QK}(\delta_1), S_{QK}(\delta_2), \dots, S_{QK}(\delta_n)$ by $\tau_1, \tau_2, \dots, \tau_n$
7: $\text{Roll}_V(\tau_1), \text{Roll}_V(\tau_2), \dots, \text{Roll}_V(\tau_n) \leftarrow$ Reorder $\text{Roll}_V(\delta_1), \text{Roll}_V(\delta_2), \dots, \text{Roll}_V(\delta_n)$ by $\tau_1, \tau_2, \dots, \tau_n$
8: $\hat{R}_{QK}(\tau_1), \hat{R}_{QK}(\tau_2), \dots, \hat{R}_{QK}(\tau_n) \leftarrow$ Compute a Softmax function on $R_{QK}(\tau_1), R_{QK}(\tau_2), \dots, R_{QK}(\tau_n)$
9: $p \leftarrow$ Compute the p -feature number by the floor product of c and $\log(N)$ in Eq (24)
10: $\hat{A} \leftarrow$ Compute the summation of product of the first p numbers of $\text{Roll}_V(\tau_1), \text{Roll}_V(\tau_2), \dots, \text{Roll}_V(\tau_n)$ and $\hat{R}_{QK}(\tau_1), \hat{R}_{QK}(\tau_2), \dots, \hat{R}_{QK}(\tau_n)$ through Eq (26)
11: **return** $\hat{A}$

4.4. Influence controlling of each cyclical output in each layer

To modulate the influence of each cyclical output within the decoder's sequence decomposition layers, a learnable projection vector for each cyclical component, W_{ij} , is introduced, where i -parameter indicates the number of heads and j -parameter the number of layers. With three sequence decomposition layers typically employed in the decoder's architecture, this results in $W_{i,1}, W_{i,2}, W_{i,3}$. The influence control is accomplished by cross-multiplying the learnable projection vector, W_{ij} , with the cyclical component of each layer, $\tilde{x}_{c(i,j)}$, as presented in Eq (27).

$$IC_{ij} = W_{ij} \times \tilde{x}_{c(i,j)} \quad (27)$$

4.5. Prediction layer

After the refining processes are finalized, the prediction layer is utilized at the end to predict the desired target. This is accomplished through the summation of the refined decomposed components, as expressed in Eq (28). The first component involves a cross-product of W_s and $\tilde{Q}_s$, where W_s represents the deeply transformed seasonal component vector corresponding to the refined $\tilde{Q}_s$. The other component is the final refined cyclical component, $\tilde{x}_c$.

$$\text{Prediction} = W_s \times \tilde{Q}_s + \tilde{x}_c \quad (28)$$

4.6. Workflow of proposed RAT approach

The step-by-step workflow of the proposed RAT approach is meticulously outlined, referencing Fig. 3. Fig. 3 demonstrates the entire process with all procedural steps predefined. The equations mentioned throughout Sections 4.1 to 4.6 for implementing the RAT approach are translated into procedural steps illustrated in Figs. 6-8. The procedural steps beginning with the letter B pertain to the initial settings for both the encoder and decoder, as seen in Fig. 6. Steps prefixed with E are specific to the encoder, depicted in Fig. 7, while D denotes steps for the decoder, showcased in Fig. 8. For simplicity, the number of heads, i , is set to one, with both the number of encoders and decoders also being one. If multiple encoders are used, steps E4 to E8 are sequentially repeated until reaching the final encoder, which must repeat steps E4 to E9. Similarly, for a multi-decoder setup, steps D7 to D13 are executed in sequence, with the final decoder required to complete steps D7 to D14. It is noted that some details, such as adjusting arguments, kernel size, and stride, are omitted in Figs. 6-8 for brevity but are considered in practice. These figures are designed to highlight the main procedures of the RAT model. The workflow begins with the initial steps under B, followed by the encoder's procedures under E, and concludes with the decoder's processes outlined under D.

4.7. A training strategy of forming TL model

TL is a critical technique in deep learning, enabling the application of knowledge from one problem domain to another, thus decreasing the data and computational resources required for training. This method uses a pre-trained model as a foundation, with fine-tuning subsequently adjusting its weights for adaptation to a new task. Such an approach not only accelerates the training process but also improves efficiency compared to starting from scratch. Fine-tuning further adapts the model for new domains or enhances its performance on specific tasks.

The architectural framework of the proposed approach is depicted in Fig. 9, identifying the source dataset as $\mathcal{D}_s$ and the target dataset as $\mathcal{D}_T$. The strategy is structured into three pivotal stages, aimed at efficiently extracting and utilizing features from the source dataset.

The first stage is pre-training, where the proposed RAT model commences its learning from the source dataset, $\mathcal{D}_s$. This phase is fundamental for acquiring a comprehensive understanding of the general

```

B1:
1: Initially, load an input time series into  $x$ .
2: Create a 1D-vector of  $W_S$ .
3: Create three 1D-vectors of  $W_{i,1}$ ,  $W_{i,2}$  and  $W_{i,3}$ .
   //  $W_{i,1}$ ,  $W_{i,2}$  and  $W_{i,3}$  are for controlling the
   influence of each current cyclical output
   in each layer and  $i$  is the number of heads.//
For encoder:
4: Create a 1D-vector of  $Q$ -weight for  $i$  head(s),
    $W_{1E}^Q, W_{2E}^Q, \dots, W_{iE}^Q$ .
5: Create a 1D-vector of  $K$ -weight for  $i$  head(s),
    $W_{1E}^K, W_{2E}^K, \dots, W_{iE}^K$ .
6: Create a 1D-vector of  $V$ -weight for  $i$  head(s),
    $W_{1E}^V, W_{2E}^V, \dots, W_{iE}^V$ .
For decoder:
7: Create a 1D-vector of  $Q$ -weight for  $i$  head(s),
    $W_{1D}^Q, W_{2D}^Q, \dots, W_{iD}^Q$ .
8: Create a 1D-vector of  $K$ -weight for  $i$  head(s),
    $W_{1D}^K, W_{2D}^K, \dots, W_{iD}^K$ .
9: Create a 1D-vector of  $V$ -weight for  $i$  head(s),
    $W_{1D}^V, W_{2D}^V, \dots, W_{iD}^V$ .
// The vectors created in the numbers of 4 to 9 are
used in the function of  $RoPE(\cdot)$  and in the MLP
layer. //

```

Fig. 6. Steps inside procedural number started with letter B.

features and patterns within the source dataset, establishing a solid foundation for subsequent stages.

The second stage is freezing. After pre-training the RAT model, the encoder layers are frozen to preserve the learned representations from the source dataset. This step prevents updates to these layers during the fine-tuning stage's backpropagation, ensuring the preservation of knowledge from the source dataset, $\mathcal{D}_s$, which facilitates the transfer of knowledge to the target dataset.

The final stage is fine-tuning, during which the RAT model undergoes fine-tuning with the target dataset, $\mathcal{D}_T$. This key phase adjusts the model's parameters to better reflect the specific features and dynamics of the target dataset, enhancing its sensitivity to the target market's nuances and improving predictive accuracy.

5. Experiments

5.1. Datasets

This study provides a comprehensive analysis of various datasets from financial markets, including indices from developed and emerging markets, cryptocurrency data, and commodity market data, as summarized in Table 4. The data, sourced from Yahoo Finance (<https://finance.yahoo.com/>). Raw data from January 1, 2017, to December 31, 2022, is collected covering each trading day's index code, daily opening, closing, highest, and lowest prices. The datasets were chronologically divided into training, validation, and test sets, adhering to a 7:1:2 ratio. In addition, all significant parameters related to the experiments are summarized in Table 5. The experiments were conducted on a laptop equipped with a 1.70 GHz Intel Core i5-12500H CPU, NVIDIA RTX 3050Ti graphics, and 16 GB RAM, adopting Pytorch 1.13.1 with a Python interpreter (Python 3.9).

5.2. Baseline models

The TL model using the proposed RAT model was evaluated against

the following baselines models:

- LSTM – Introduced by Hochreiter et al. in 1997 (He et al., 2019), the LSTM model incorporates input, output, and forget gates. It is adept at addressing long-term dependencies in data, proving to be an effective tool for time-series forecasting tasks.
- CNN-LSTM – This model combines the strengths of CNN and LSTM, showcasing groundbreaking outcomes (Lu et al., 2020). The CNN extracts features from the input data, which are then flattened and processed by the LSTM layer to analyze temporal characteristics.
- GRU – Simplifying the LSTM architecture, the GRU model merges the forget and input gates into an update gate and combines the hidden state with the cell state for more efficient computation (Chung et al., 2014).
- Transformer – Introduced by Vaswani et al. in 2017, the Transformer model has significantly impacted deep learning, especially in NLP (Vaswani et al., 2017).

5.3. Evaluation metrics

The performance of the proposed RAT model and the baseline models was assessed using three widely recognized metrics, mentioned in Section 5.2, as follows:

5.3.1. Mean squared error

It is calculated as represented in Eq (29), where $\hat{y}_i$ represents the predicted value, y_i denotes the ground-truth, and n is the length of the entire sequence. The MSE metric averages the squared differences between predicted and actual values, making it highly sensitive to the magnitude of prediction errors.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (29)$$

Encoder

```

// SeqDecomp(·) run by Algorithm 1 //
E1:
1:  $Q_{enc}, K_{enc} \leftarrow$  Plug  $x$  into  $RoPE(\cdot)$  run by Algorithm 2.
E2:
1:  $x_{em} \leftarrow$  Perform the input embedding by
   1D CNNs on  $x$ .
E3:
1:  $x_{pe} \leftarrow$  Perform the positional encoding using
   Eqs (8)-(9) on  $x_{em}$ .
2:  $V_{enc} \leftarrow$  Project  $x_{pe}$  onto  $V$  using Eq (6).
E4:
1:  $\hat{A}_{enc} \leftarrow$  Plug  $Q_{enc}, K_{enc}, V_{enc}$  into
    $Autocorrelation(\cdot)$  run by Algorithms 4 and 5.
2:  $Q_{enc} \leftarrow$  Sum  $\hat{A}_{enc}$  with  $Q_{enc}$ .
3:  $K_{enc} \leftarrow$  Sum  $\hat{A}_{enc}$  with  $K_{enc}$ .
4:  $V_{enc} \leftarrow$  Sum  $\hat{A}_{enc}$  with  $V_{enc}$ .
E5:
1:  $Q_{sen} \leftarrow$  Plug  $Q_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
2:  $K_{sen} \leftarrow$  Plug  $K_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
3:  $V_{sen} \leftarrow$  Plug  $V_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
E6:
1:  $Q_{enc} \leftarrow$  Perform 1D CNNs through MLP on  $Q_{sen}$ .
2:  $K_{enc} \leftarrow$  Perform 1D CNNs through MLP on  $K_{sen}$ .
3:  $V_{enc} \leftarrow$  Perform 1D CNNs through MLP on  $V_{sen}$ .
E7:
1:  $Q_{enc} \leftarrow$  Sum  $Q_{enc}$  with  $Q_{sen}$ .
2:  $Q_{enc} \leftarrow$  Do the normalization in Eq (7)
   using  $Q_{enc}$  as Input.
3:  $K_{enc} \leftarrow$  Sum  $K_{enc}$  with  $K_{sen}$ .
4:  $K_{enc} \leftarrow$  Do the normalization in Eq (7)
   using  $K_{enc}$  as Input.
5:  $V_{enc} \leftarrow$  Sum  $V_{enc}$  with  $V_{sen}$ .
6:  $V_{enc} \leftarrow$  Do the normalization in Eq (7)
   using  $V_{enc}$  as Input.
E8:
1:  $Q_{sen} \leftarrow$  Plug  $Q_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
2:  $K_{sen} \leftarrow$  Plug  $K_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
3:  $V_{sen} \leftarrow$  Plug  $V_{enc}$  into  $SeqDecomp(\cdot)$  and
   filter out the cyclical component.
E9:
1:  $K_{enc}, V_{enc} \leftarrow$  Store  $K_s$  and  $V_s$  into  $K_{enc}$  and  $V_{enc}$ ,
   respectively, and transfer
    $K_{enc}$  and  $V_{enc}$  to the decoder.

```

Fig. 7. Steps inside procedural number started with letter E.

5.3.2. Mean absolute error

MAE is formulated in Eq (30), where $\hat{y}_i$ represents the predicted value, y_i denotes the ground-truth, and n is the sequence's length. The MAE averages the absolute differences between predictions and actual values, ensuring errors do not negate each other. MAE applies equal weighting to each difference in the overall average.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (30)$$

5.3.3. R-square (R^2)

R^2 is computed using Eq (31), assessing the proportion of variance in the dependent variable explained by the independent variables within

the model. R^2 values range from 0 to 1, with higher values indicating a better model fit.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum ((y_i - \bar{y}))^2} \quad (31)$$

5.4. Experimental setup

The experiments were carried out on 12 datasets, each comprising daily historical trading data. The LSTM, CNN-LSTM, GRU, Transformer, and RAT models were used to predict stock prices for the subsequent five days across these markets. The models' predictive performance was evaluated using MSE, MAE, and R^2 metrics. The specific experimental

Decoder

D1:

1: $x_s, x_c \leftarrow$ Plug x into $SeqDecomp(\cdot)$ run by Algorithm 1.

D2:

1: $x_s \leftarrow$ Pad a zero value at the end with a number of a prediction length.

D3:

1: $x_c \leftarrow$ Pad a mean value of x at the end with a number of a prediction length.

D4:

1: $Q_{dec}, K_{dec} \leftarrow$ Plug x_s into $RoPE(\cdot)$ run by Algorithm 2.

D5:

1: $x_{sem} \leftarrow$ Perform the input embedding by 1D CNNs on x_s .

D6:

1: $x_{spe} \leftarrow$ Perform the positional encoding using Eqs (8)-(9) on x_{sem} .2: $V_{dec} \leftarrow$ Project x_{spe} onto V using Eq (6).

D7:

1: $\hat{A}_{dec} \leftarrow$ Plug $Q_{dec}, K_{dec}, V_{dec}$ into $Autocorrelation(\cdot)$ run by Algorithms 4 and 5.2: $Q_{dec} \leftarrow$ Sum $\hat{A}_{dec}$ with Q_{dec} .3: $K_{dec} \leftarrow$ Sum $\hat{A}_{dec}$ with K_{dec} .4: $V_{dec} \leftarrow$ Sum $\hat{A}_{dec}$ with V_{dec} .

D8:

1: $Q_{sdec}, Q_{cdec1} \leftarrow$ Plug Q_{dec} into $SeqDecomp(\cdot)$ run by Algorithm 1.2: $K_{sdec}, K_{cdec} \leftarrow$ Plug K_{dec} into $SeqDecomp(\cdot)$ run by Algorithm 1.3: $V_{sdec}, V_{cdec} \leftarrow$ Plug V_{dec} into $SeqDecomp(\cdot)$ run by Algorithm 1.4: $Q_{cdec1} \leftarrow$ Do a cross product of Q_{cdec1} and $W_{1,1}$ in Eq (27).5: $\tilde{x}_c \leftarrow$ Sum Q_{cdec1} with x_c in Eq (27).6: $K_{cdec} \leftarrow$ Sum K_{cdec} with x_c .7: $V_{cdec} \leftarrow$ Sum V_{cdec} with x_c .

D9:

1: Acquire K_{enc}, V_{enc} 2: $\hat{A}_{dec} \leftarrow$ Plug $Q_{sdec}, K_{enc}, V_{enc}$ into $Autocorrelation(\cdot)$ run by Algorithms 4 and 5.3: $Q_{dec} \leftarrow$ Sum $\hat{A}_{dec}$ with Q_{sdec} .

D10:

1: $Q_{sdec}, Q_{cdec2} \leftarrow$ Plug Q_{dec} into $SeqDecomp(\cdot)$ run by Algorithm 1.2: $Q_{cdec2} \leftarrow$ Do a cross product of Q_{cdec2} and $W_{1,2}$ in Eq (27).3: $\tilde{x}_c \leftarrow$ Sum Q_{cdec2} with $\tilde{x}_c$ in Eq (27).

D11:

1: $Q_{dec} \leftarrow$ Perform 1D CNNs through MLP on Q_{sdec} .

D12:

1: $Q_{dec} \leftarrow$ Sum Q_{enc} with Q_{sdec} .2: $Q_{dec} \leftarrow$ Do the normalization in Eq (7) using Q_{dec} as Input.

D13:

1: $\tilde{Q}_{sdec}, Q_{cdec3} \leftarrow$ Plug Q_{dec} into $SeqDecomp(\cdot)$ run by Algorithm 1.2: $Q_{cdec3} \leftarrow$ Do a cross product of Q_{cdec3} and $W_{1,3}$ in Eq (27).3: $\tilde{x}_c \leftarrow$ Sum Q_{cdec3} with $\tilde{x}_c$ in Eq (27).

D14:

1: $\tilde{Q}_{dec} \leftarrow$ Do a cross product of $\tilde{Q}_{sdec}$ and W_s in Eq (28).2: Prediction $\leftarrow$ Sum $\tilde{Q}_{dec}$ with $\tilde{x}_c$ in Eq (28).

Fig. 8. Steps inside procedural number started with letter D.

process for the RAT model included:

The experiment focused on predicting financial market prices based on historical trading data, starting with data preparation and loading, including open, close, high, and low prices. The dataset was divided into training, validation, and testing subsets, with 70 % of the data allocated for training, 10 % for validation, and 20 % for testing. Data were segmented into 15-day windows, where the initial 10 days served as input and the subsequent five days as the prediction target. A Standard Scaler was applied to normalize the data, ensuring each feature across the input sequence had a mean of zero and a standard deviation of one. The RAT model was configured with two encoders and one decoder, each featuring an autocorrelation mechanism for efficient temporal sequence processing. Training began with the model using the Adam optimizer and MSE loss for 50 epochs, incorporating forward and backward passes and parameter optimization. Early stopping and learning rate adjustments were employed to prevent overfitting and

ensure model convergence. The model's predictive performance was then evaluated on the test set using MSE, MAE, and R2 metrics, providing a thorough assessment of its forecasting capabilities for financial market prices.

Subsequently, the DJIA was utilized for pre-training the LSTM, CNN-LSTM, GRU, Transformer, and RAT models. The remaining 10 datasets, excluding the S&P 500 index, were designated as target datasets. These pre-trained models underwent TL on the target datasets to evaluate their generalization capabilities. For the RAT model, the encoder was frozen, and only the decoder underwent fine-tuning on the target-domain dataset.

Among the major U.S. stock indices, the NASDAQ composite is primarily composed of technology and internet stocks, providing a partial reflection of the economy. The S&P 500, though more comprehensive, may introduce variability due to its diversity. The Dow Jones Industrial Average, comprising 30 diverse blue-chip companies, offers a more

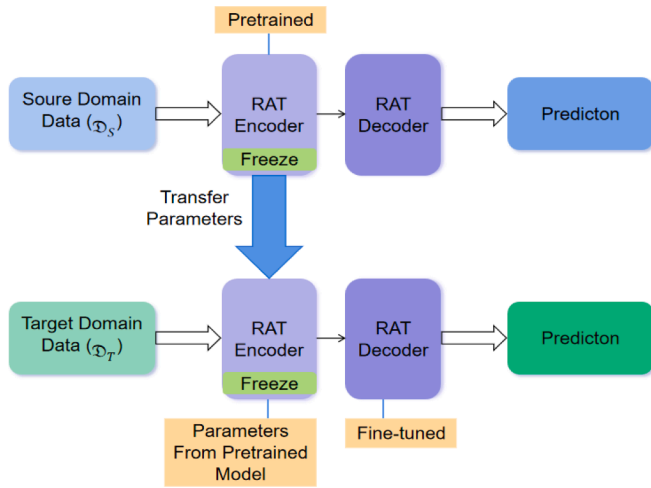


Fig. 9. Procedural flow of TL strategy.

Table 4
Datasets used in experiments.

Group	Full name	Short name	Frequency	Sample size
Developed Market	Dow Jones	DJIA	daily	1511
	Industrial Average	S&P500	daily	1511
	Standard & Poor's 500 Index	NASDAQ	daily	1511
	National Association of Securities Dealers Automated Quotations	N225	daily	1466
	Nikkei 225	DAX 30	daily	1521
	Deutscher Aktien Index	BSE	daily	1480
	the S&P Bombay Stock Exchange Sensitive Index	IBOVESPA	daily	1486
Emerging Market	Ibovespa Brasil Sao Paulo Stock Exchange Index	SET	daily	1454
	Thailand SET Index	BTC	daily	2191
Cryptocurrency Market	Bitcoin	LTC	daily	2191
	Litcoin	GOLD	daily	1511
Commodity Market	Gold	OIL	daily	1513
	Crude Oil			

representative snapshot of the U.S. economy. Therefore, using the Dow Jones for pre-training and TL presents an optimal balance between diversity and representational stability.

6. Results and discussion

6.1. Results on different financial markets prediction

In a study focused on financial forecasting, the performance of LSTM, CNN-LSTM, GRU, Transformer, and RAT models was employed, with each employing 10 days of historical data to predict the prices for the next five days.

The multivariate financial time-series forecasting results for the 12 datasets are shown in Table 6. The proposed RAT model notably boosts prediction performance across all datasets, notably on the NASDAQ and BTC datasets, which are characterized by their lack of clear periodicity. This improvement is attributed to the RAT model's effective capture of temporal dependencies through its autocorrelation and rotary-positional encoding mechanisms.

When compared to the state-of-the-art Transformer model (Wang et al., 2022); the RAT model demonstrates noticeable improvements in

all three evaluation metrics across 11 datasets. Notably, the RAT model achieves an 84.90 % (0.312→0.0471) MSE reduction for the BSE SENSEX, 72.43 % (0.6939→0.1913) for OIL, 60 % (0.1471→0.0584) for BTC, and 57.85 % (0.1599→0.0674) for the S&P500. Furthermore, it records R^2 increases of 259.47 % for the BSE SENSEX and 76.94 % for OIL, indicating a superior ability to extract dependencies in time-series data, particularly trends and cyclical patterns, compared to self-attention mechanisms. Unlike traditional positional encodings that provide absolute positional information, rotary-positional encoding preserves relative positioning, enhancing the ability of the model to recognize and exploit distance-related patterns in time series.

The RAT model outperforms the LSTM, CNN-LSTM, and GRU models across all evaluated datasets, showing considerably lower MSE and MAE values and notably higher R^2 values. This suggests that while LSTM and GRU are designed to solve the vanishing gradient issue of RNNs for long-term dependencies, they still face challenges with long sequences due to information decay over time. Similarly, the CNN-LSTM model faces challenges in capturing long-term dependencies as its effectiveness relies on LSTM. Conversely, the RAT model, with its autocorrelation mechanism, adeptly addresses entire sequence lengths, capturing long-term dependencies without succumbing to information decay.

Comparisons of the prediction results with ground-truth values for the RAT and baseline models reveal significant insights, as illustrated in Fig. 10. LSTM, CNN-LSTM, and GRU, typically used for one-step forecasting, encounter significant error accumulation when applied to dynamic forecasting. This dynamic forecasting challenge results in substantial forecasting bias and elevated MSE due to the cumulative effect of previous prediction errors. Additionally, the complexity of financial time-series data poses additional challenges for these models, especially given the constraints of computer memory. Without a heuristic selection mechanism, memory cells are unable to efficiently capture global patterns, instead remembering only restricted patterns that disrupt the forecasting process. A notable problem with these models is the time-lag phenomenon, where time delays introduce errors despite their partial success in capturing fluctuations. The Transformer model, with its self-attention mechanism, exhibits only minor time delays, due to its capacity to effectively capture globally significant fluctuation features. Remarkably, the GRU model shows remarkable improvement over both LSTM and CNN-LSTM in prediction accuracy, achieving a substantial reduction in MSE. For some datasets, its performance is on par with the Transformer, because of GRU's simplified gating mechanism, which combines the input and forget gates into a single update gate. This simplification reduces the model's parameter count, streamlining the training process. Such efficiency not only accelerates the learning process of the model but also facilitates quick adaptation to the dynamic nature of financial markets. The Transformer model's

Table 5
Parameter settings for all models.

Model	Parameters
RAT	$d_{model} = 512$, learning rate = 0.0001, seq-len = 10, pred-len = 5, Encoder layers = 2, decoder layers = 1, multi-heads = 4, loss function = mse, Batch size = 32, optimizer = Adam, train_epochs = 50
Transformer	$d_{model} = 512$, learning rate = 0.0001, seq-len = 10, pred-len = 5, Encoder layers = 2, decoder layers = 1, multi-heads = 4, loss function = mse, Batch size = 32, optimizer = Adam, train_epochs = 50
GRU	seq-len = 10, pred-len = 5, GRU layer = 2, loss function = mse, learning rate = 0.001, Batch size = 256, optimizer = Adam, train_epochs = 100
CNN-LSTM	seq-len = 10, pred-len = 5, CNN layer = 1, LSTM layer = 2, loss function = mse, learning rate = 0.001, Batch size = 256, optimizer = Adam, train_epochs = 100
LSTM	seq-len = 10, pred-len = 5, LSTM layer = 2, loss function = mse, learning rate = 0.001, Batch size = 256, optimizer = Adam, train_epochs = 100

generalizability for price prediction across different financial markets has its limitations, performing well in specific markets such as IBOVESPA but faltering in others with different types and trends, such as BSE SENSEX. Moreover, the sparse point-wise self-attention in the Transformer model leads to imprecisions, particularly in capturing local features and crucial details including minor fluctuations and inflection points.

The RAT model, however, closely aligns with ground-truth and demonstrates the best fit among the models analyzed. Through its sequence decomposition and autocorrelation, the RAT model effectively captures cyclical and trend features across different markets. Unlike the traditional Transformer, which focuses on singular time points, the RAT model's autocorrelation covers broader periods, enhancing its ability to manage continuous inductive bias and minimize information loss during training. Consequently, the RAT model accurately captures vital details such as minor fluctuations and inflection points, reflecting a deeper understanding of the data. Moreover, it demonstrates strong generalization capabilities, maintaining stable and high predictive accuracy across all markets, including the crude oil market, known for its black swan events and extreme outliers.

6.2. TL ability

In practical forecasting and trading tasks, each financial market is defined by unique fundamentals and characteristics that dictate its trading patterns. Training neural networks on every conceivable stock, index, currency, or commodity is impractical. Therefore, the ability of neural networks to apply TL, predicting the behavior of unlearned financial instruments based on recognized patterns in trained data, is crucial.

A comparison of baseline and RAT models, each initially trained on the DJIA and subsequently adapted through TL to various financial markets, is shown in Table 7. This TL challenge involves adapting to different market environments, such as transitioning from American to Japanese markets, and shifting price categories, from market indices to commodities. Different markets are marked by varied trading rules and participants, contributing to variability and complex predictions. The lower the correlation between two underlying assets, the more challenging the TL task becomes. However, the RAT model's exceptional TL capabilities shine through. It consistently outperforms the other models, achieving higher R^2 scores that indicate a stronger correlation between its predictions and actual values. Moreover, the RAT model significantly lowers the MSE, enhancing prediction accuracy, and reduces the MAE compared to baseline models, suggesting a lower error rate and enhanced forecasting proficiency. These results highlight the RAT model's exceptional precision, reliability, and generalizability across diverse financial markets when augmented by TL.

A detailed examination of TL effects across different models reveals their performance before and after applying TL, see Figs. 11, 12, and 13. The LSTM, CNN-LSTM, and Transformer models largely failed to achieve TL gains across most datasets. This shortfall stems from their high expressiveness and ability of capturing complex data patterns, which, paradoxically, also predisposes them to overfitting. Consequently, when these models, pre-trained on one dataset, are transferred to another, they may import biases or specific patterns from the source dataset that prove irrelevant or even misleading for the target dataset. Conversely, the GRU model displayed TL gains in six datasets and exhibited a negative transfer in four datasets. Its relative simplicity, with fewer parameters than its counterparts, reduces the risk of overfitting, particularly in small and diverse datasets, making it more suitable for TL. This simplicity ensures the GRU is less likely to internalize noise from the training data, thus fostering better generalization from the source to the target domain and enhancing its TL efficacy. The success of the RAT model in TL across all datasets, surpassing other models, is primarily attributed to its innovative use of rotary-positional encoding. This method of encoding relative positions significantly enhances the

Table 6

Experiment results for different forecasting methods.

Dataset	Networks	MSE	MAE	R^2
DJIA	LSTM	0.1464	0.3093	0.6297
	CNN-LSTM	0.1306	0.2976	0.6696
	GRU	0.0877	0.2428	0.7782
	Transformer	0.0724	0.2129	0.8427
S&P500	RAT	0.0604	0.1872	0.8686
	LSTM	0.2855	0.4385	0.6673
	CNN-LSTM	0.2790	0.4356	0.6750
	GRU	0.1476	0.3230	0.8280
NASDAQ	Transformer	0.1599	0.3197	0.7829
	RAT	0.0674	0.2010	0.9085
	LSTM	0.1381	0.3105	0.8302
	CNN-LSTM	0.1254	0.2945	0.8458
N225	GRU	0.0890	0.2446	0.8906
	Transformer	0.0861	0.2377	0.8872
	RAT	0.0409	0.1595	0.9464
	LSTM	0.1280	0.2722	0.2112
DAX 30	CNN-LSTM	0.1469	0.3029	0.0948
	GRU	0.0931	0.2324	0.4263
	Transformer	0.0864	0.2255	0.5526
	RAT	0.0688	0.1976	0.6440
BSE SENSEX	LSTM	0.2914	0.4157	0.7849
	CNN-LSTM	0.3063	0.4116	0.7740
	GRU	0.2342	0.3682	0.8272
	Transformer	0.1553	0.2907	0.9002
IBOVESPA	RAT	0.1423	0.2760	0.9086
	LSTM	0.1653	0.3235	0.4022
	CNN-LSTM	0.1677	0.3108	0.3934
	GRU	0.0767	0.2270	0.7226
SET	Transformer	0.3120	0.4631	-0.5231
	RAT	0.0471	0.1673	0.8342
	LSTM	0.0508	0.1824	0.5382
	CNN-LSTM	0.0502	0.1820	0.5438
BTC	GRU	0.0448	0.1698	0.5930
	Transformer	0.0291	0.1358	0.7362
	RAT	0.0258	0.1281	0.7656
	LSTM	0.0273	0.1324	0.5191
LTC	CNN-LSTM	0.0237	0.1262	0.5820
	GRU	0.0247	0.1279	0.5656
	Transformer	0.0174	0.0995	0.7022
	RAT	0.0174	0.1008	0.7022
GOLD	LSTM	0.2961	0.4450	0.8906
	CNN-LSTM	0.2389	0.3715	0.9117
	GRU	0.1299	0.2440	0.9520
	Transformer	0.1471	0.2723	0.9440
OIL	RAT	0.0584	0.1655	0.9777
	LSTM	0.0533	0.1460	0.9236
	CNN-LSTM	0.0505	0.1401	0.9276
	GRU	0.0492	0.1446	0.9295
	Transformer	0.0447	0.1453	0.9464
	RAT	0.0312	0.1106	0.9626
	LSTM	0.0415	0.1648	0.7253
	CNN-LSTM	0.03691	0.1546	0.7556
	GRU	0.03023	0.1385	0.7998
	Transformer	0.0192	0.1089	0.8533
	RAT	0.0163	0.0982	0.8755
	LSTM	0.6758	0.6252	-0.5265
	CNN-LSTM	0.6851	0.6401	-0.5474
	GRU	0.2308	0.3650	0.4788
	Transformer	0.6939	0.6089	0.4849
	RAT	0.1913	0.3248	0.8580

model's understanding of temporal dynamics in time-series data, facilitating efficient capture and utilization of sequence order and thereby enhancing accuracy across diverse time scales. Additionally, the RAT model's versatility in adapting to various temporal patterns, unrestricted by sequence length or specific features, makes it highly versatile and effective for TL. Moreover, the autocorrelation mechanism within the RAT model excels at capturing long-term dependencies and periodic patterns in the data, enabling the model to assimilate more general time-series features that are transferable and valuable across different application scenarios.

It is noteworthy that all baseline models struggled to accurately

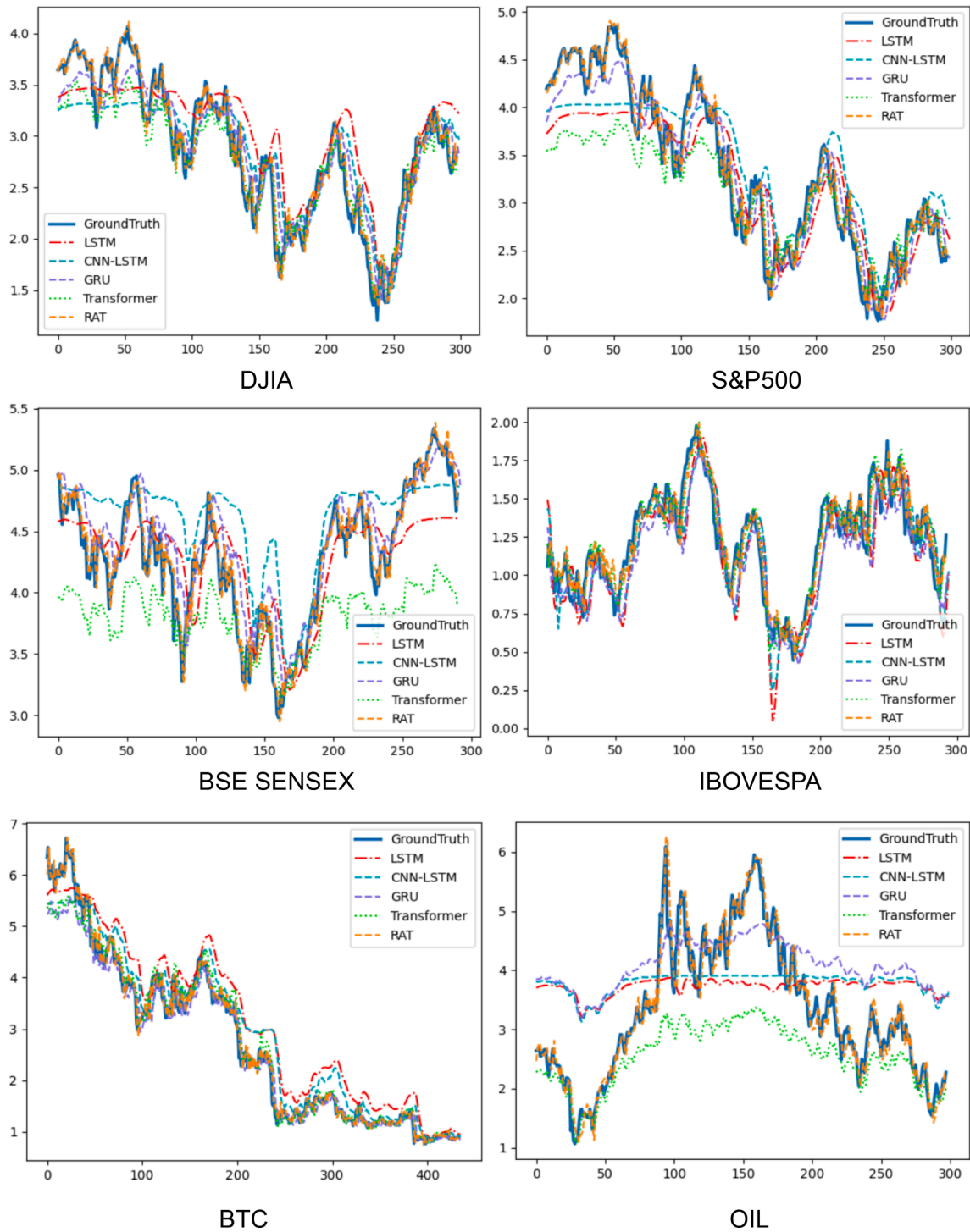


Fig. 10. Comparison of LSTM, Transformer, and RAT prediction performance.

predict oil prices, a difficulty largely attributable to the extraordinary event on April 20, 2020, when U.S. oil prices plummeted to an unprecedented negative \$37.63 per barrel. Despite this anomaly, the RAT model demonstrated commendable performance on the oil dataset, achieving MSE, MAE, and R2 values of 0.1913, 0.3248, and 0.8580, respectively. The application of TL further improved these results, decreasing the MSE and MAE to 0.1882 and 0.3217, respectively, and increasing R2 to 0.8603. The RAT model's resilience in navigating such market irregularities underscores its robustness, a critical attribute in the volatile financial market environment.

6.3. Ablation study

The performance comparison between RAT models with and without rotary-positional encoding. The experimental results are shown in Table 8 and Table 9. The analysis indicates the former's superior predictive capability across all markets. This advantage is consistently observed in both initial predictions and those post-TL, with the rotary-positional encoding-enhanced RAT model yielding lower MSE and MAE, and higher R2 values. The introduction of rotary-positional encoding not only enhances the model's predictive accuracy and reliability across various financial markets but also underscores its

Table 7
Experiment results for TL.

Dataset	TL Networks	MSE	MAE	R ²
NASDAQ	LSTM	0.3018	0.4567	0.8246
	CNN-LSTM	0.1356	0.3001	0.8392
	GRU	0.0781	0.2315	0.9040
	Transformer	0.0801	0.2278	0.8951
	RAT	0.0407	0.1586	0.9467
N225	LSTM	0.1265	0.2728	0.2207
	CNN-LSTM	0.0999	0.2410	0.3846
	GRU	0.1021	0.2459	0.3709
	Transformer	0.0841	0.2196	0.5646
	RAT	0.0683	0.1962	0.6466
DAX 30	LSTM	0.2900	0.4158	0.7859
	CNN-LSTM	0.3139	0.4173	0.7683
	GRU	0.2243	0.3674	0.8345
	Transformer	0.1847	0.3214	0.8813
	RAT	0.1416	0.2757	0.9090
BSE SENSEX	LSTM	0.2184	0.3569	0.2100
	CNN-LSTM	0.1737	0.3187	0.3715
	GRU	0.0762	0.2260	0.7245
	Transformer	0.4646	0.6039	−0.6356
	RAT	0.0469	0.1669	0.8350
IBOVESPA	LSTM	0.0547	0.1902	0.5028
	CNN-LSTM	0.0510	0.1819	0.5366
	GRU	0.0424	0.1677	0.6141
	Transformer	0.0302	0.1384	0.7264
	RAT	0.0253	0.1255	0.7708
SET	LSTM	0.0258	0.1290	0.5452
	CNN-LSTM	0.0251	0.1252	0.5582
	GRU	0.0239	0.1223	0.5794
	Transformer	0.0160	0.0980	0.7254
	RAT	0.0172	0.1007	0.7055
BTC	LSTM	0.3726	0.5005	0.8623
	CNN-LSTM	0.2795	0.4080	0.8967
	GRU	0.1392	0.2682	0.9486
	Transformer	0.1290	0.2519	0.9509
	RAT	0.0582	0.1655	0.9778
LTC	LSTM	0.0549	0.1440	0.9212
	CNN-LSTM	0.0536	0.1441	0.9231
	GRU	0.0547	0.1519	0.9216
	Transformer	0.0483	0.1377	0.9421
	RAT	0.0309	0.1106	0.9629
GOLD	LSTM	0.0323	0.1451	0.7862
	CNN-LSTM	0.0348	0.1486	0.7696
	GRU	0.0284	0.1335	0.8117
	Transformer	0.0197	0.1107	0.8495
	RAT	0.0161	0.0980	0.8765
OIL	LSTM	0.8281	0.6774	−0.8705
	CNN-LSTM	0.6090	0.5825	−0.3756
	GRU	0.2532	0.3813	0.4281
	Transformer	0.7681	0.6361	0.4299
	RAT	0.1882	0.3217	0.8603

significant role in improving the overall effectiveness of the RAT model in financial forecasting. This experiment highlights the substantial influence of rotary-positional encoding in enhancing the model's performance, showcasing its indispensability in crafting robust, high-performing predictive models.

Rotary-positional encoding significantly improves the model's ability to discern relative position relationships within sequences while retaining absolute position information. Figs. 14 and 15 compare the RAT model's prediction results without and with rotary-positional encoding, respectively, against the ground-truth values for the DJIA dataset. The inclusion of rotary-positional encoding effectively enhances the model's proficiency in capturing the temporal and spatial characteristics of financial time series. This enhancement is crucial for accurately forecasting essential details such as minor fluctuations and inflection points, further affirming the value of rotary-positional encoding in financial market prediction tasks.

7. Conclusion

To improve the ability of the model to capture local dependencies in financial time series and enhance its generalization across different financial markets, the RAT model was developed. This approach leverages deep TL to migrate trained components from the source domain to the target domain. Autocorrelation is employed within this model to more effectively capture periodic features in time-series data, replacing the sparse point-wise self-attention mechanism. Additionally, rotary-positional encoding is used instead of traditional positional encoding to preserve the relative positions within a sequence, thereby enriching the model's trend analysis capabilities. In evaluations conducted on 12 different datasets, the RAT model consistently surpassed advanced baseline models, such as LSTM and Transformers, in accuracy with approximately 35.83 % reduction in error. Moreover, the application of the pre-trained RAT model through TL yielded further improvements in predictive accuracy across all datasets. Despite RAT's impressive performance, it encounters certain limitations: (1) The data type explored in this study is limited to one-dimensional (1D) time series of textual



Fig. 11. MSE values of compared networks and their TL models on different datasets.

modality, restricting the model’s learning scope to a narrow range of features. (2) A single-source TL strategy is employed, constraining the pre-trained model to acquiring and transferring behaviors from only one market. Given these limitations, future research directions are proposed:

(1) To overcome the representation limitations of 1D time series, it is suggested to explore the conversion of 1D time series into a set of 2D tensors by leveraging multiple periods. This transformation aims to incorporate intra- and inter-period variations into the 2D tensors’



Fig. 12. MAE values of compared networks and their TL models on different datasets.



Fig. 13. R^2 values of compared networks and their TL models on different datasets.

Table 8

Comparison of using and not using rotary-positional encoding.

	RAT (without rotary-positional encoding)			RAT		
	MSE	MAE	R ²	MSE	MAE	R ²
DJIA	0.0640	0.1943	0.8609	0.0604	0.1872	0.8686
S&P500	0.0711	0.2074	0.9035	0.0674	0.2010	0.9085
NASDAQ	0.0410	0.1594	0.9463	0.0409	0.1595	0.9464
N225	0.0705	0.2015	0.6349	0.0688	0.1976	0.6440
DAX 30	0.1447	0.2816	0.9070	0.1423	0.2760	0.9086
BSE SENSEX	0.0475	0.1705	0.8329	0.0471	0.1673	0.8342
IBOVESPA	0.0263	0.1271	0.7613	0.0258	0.1281	0.7656
SET	0.0177	0.1021	0.6974	0.0174	0.1008	0.7022
BTC	0.0595	0.1649	0.9773	0.0584	0.1655	0.9777
LTC	0.0338	0.1153	0.9594	0.0312	0.1106	0.9626
GOLD	0.0166	0.0994	0.8734	0.0163	0.0982	0.8755
OIL	0.2110	0.3390	0.8434	0.1913	0.3248	0.8580

Table 9

Comparison of using and without using rotary-positional encoding in TL.

	Transfer Learning (RAT without rotary-positional encoding)			Transfer Learning (RAT)		
	MSE	MAE	R ²	MSE	MAE	R ²
DJIA	/	/	/	/	/	/
S&P500	/	/	/	/	/	/
NASDAQ	0.0409	0.1598	0.9464	0.0407	0.1586	0.9467
N225	0.0691	0.1992	0.6422	0.0683	0.1962	0.6466
DAX 30	0.1397	0.2737	0.9102	0.1416	0.2757	0.9090
BSE SENSEX	0.0483	0.1729	0.8300	0.0469	0.1669	0.8350
IBOVESPA	0.0262	0.1263	0.7624	0.0253	0.1255	0.7708
SET	0.0173	0.1008	0.7041	0.0172	0.1007	0.7055
BTC	0.0584	0.1655	0.9777	0.0582	0.1655	0.9778
LTC	0.0321	0.1119	0.9615	0.0309	0.1106	0.9629
GOLD	0.0165	0.0992	0.8740	0.0161	0.0980	0.8765
OIL	0.2048	0.3321	0.8480	0.1882	0.3217	0.8603

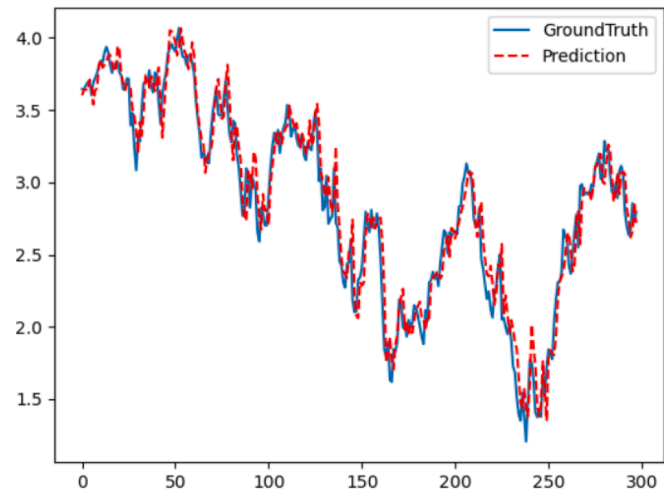


Fig. 14. Prediction performance of RAT without rotary-positional encoding on DJIA.

columns and rows, respectively. allowing the modeling of 2D variations using 2D kernels. (2) Enhancing the TL strategy through a multi-source approach could significantly improve the model's generalization capabilities. By enabling the pre-trained model to assimilate features from a variety of financial markets, this strategy aims to improve a more robust and versatile predictive model.

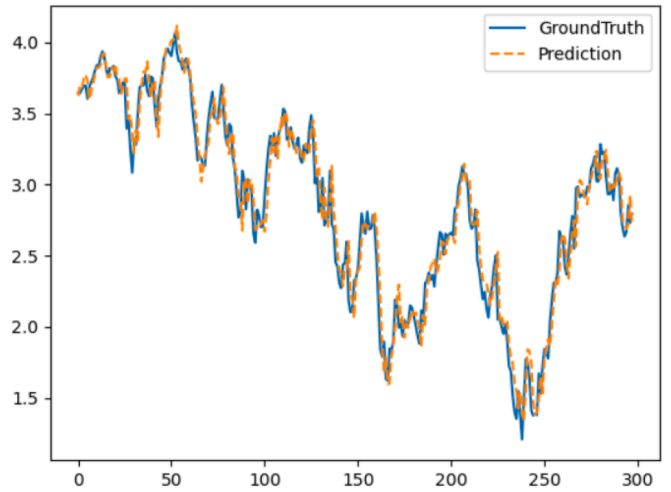


Fig. 15. Prediction performance of RAT on DJIA.

CRediT authorship contribution statement

Ling Feng: Conceptualization, Methodology, Software, Formal analysis, Resources, Data curation, Writing - original draft, Visualization. **Ananta Sinchai:** Conceptualization, Methodology, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review and editing, Supervision, Project administration.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: [Insert text provided in the DoCI form].

Acknowledgments

We would like to thank Editage (www.editage.com) for English language editing.

References

- Bahdanau, D., Cho, K., Bengio, Y., 2015. Neural machine translation by jointly learning to align and translate. In: 3rd International Conference on Learning Representations, ICLR.
- Bastos, J.A., 2022. Predicting credit scores with boosted decision trees. *Forecasting* 4 (4), 925–935.
- Chen, D., Zou, Y., Harimoto, K., Bao, R., Ren, X., Sun, X., 2019. Incorporating fine-grained events in stock movement prediction. *arXiv preprint arXiv:1910.05078*.
- Chung, J., Gulcehre, C., Cho, K., Bengio, Y., 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.
- D'Angelo, G., Palmieri, F., Robustelli, A., Castiglione, A., 2021. Effective classification of android malware families through dynamic features and neural networks. *Connect. Sci.* 33 (3), 786–801.
- Dondurur, D., 2018. Acquisition and processing of marine seismic data. Elsevier.
- Feng, F., Chen, H., He, X., Ding, J., Sun, M., Chua, T.S., 2018. Enhancing stock movement prediction with adversarial training. *arXiv preprint arXiv:1810.09936*.
- Fischer, T., Krauss, C., 2018. Deep learning with long short-term memory networks for financial market predictions. *Eur. J. Oper. Res.* 270 (2), 654–669.
- Fukushima, K., 1980. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.* 36 (4), 193–202.
- Fukushima, K., 1987. Neural network model for selective attention in visual pattern recognition and associative recall. *Appl. Opt.* 26 (23), 4985–4992.
- Fukushima, K., Imagawa, T., 1993. Recognition and segmentation of connected characters with selective attention. *Neural Netw.* 6 (1), 33–41.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep learning. MIT Press.
- Gupta, G., Katarya, R., 2021. Research on understanding the effect of deep learning on user preferences. *Arab. J. Sci. Eng.* 46, 3247–3286.
- Gupta, P., Malhotra, P., Vig, L., Shroff, G., 2018. Transfer learning for clinical time series analysis using recurrent neural networks. *arXiv preprint arXiv:1807.01705*.
- He, Q.Q., Pang, P.C.I., Si, Y.W., 2019. Transfer learning for financial time series forecasting. In: *PRICAI 2019: Trends in Artificial Intelligence*, 16th Pacific Rim International Conference on Artificial Intelligence, pp. 24–36.

- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780.
- Judd, C.H., 1908. The relation of special training to general intelligence. *Educ. Rev.* 36, 28–42.
- Kim, H.Y., Won, C.H., 2018. Forecasting the volatility of stock price index: a hybrid model integrating LSTM with multiple GARCH-type models. *Expert Syst. Appl.* 103, 25–37.
- Köksal, A., Özgür, A., 2021. Twitter dataset and evaluation of transformers for Turkish sentiment analysis. In: 2021 29th Signal Processing and Communications Applications Conference (SIU), pp. 1–4.
- Laptev, N., Yu, J., Rajagopal, R., 2018. Reconstruction and regression loss for time-series transfer learning. In: the Special Interest Group on Knowledge Discovery and Data Mining (SIGKDD) and the 4th Workshop on the Mining and Learning from Time Series (MiLeTS).
- Larochelle, H., Hinton, G.E., 2010. Learning to combine foveal glimpses with a third-order boltzmann machine. In: *Advances in Neural Information Processing Systems* 23.
- Le Meur, O., Le Callet, P., Barba, D., Thoreau, D., 2006. A coherent computational approach to model bottom-up visual attention. *IEEE Trans. Pattern Anal. Mach. Intell.* 28 (5), 802–817.
- Li, Y., Dai, H.N., Zheng, Z., 2022. Selective transfer learning with adversarial training for stock movement prediction. *Connect. Sci.* 34 (1), 492–510.
- Li, Y., Pan, Y., 2022. A novel ensemble deep learning model for stock prediction based on stock prices and news. *Int. J. Data S. Analytics.*
- Li, C., Qian, G., 2022. Stock price prediction using a frequency decomposition based GRU transformer neural network. *Appl. Sci.* 13 (1), 222.
- Lu, W., Li, J., Li, Y., Sun, A., Wang, J., 2020. A CNN-LSTM-based model to forecast stock prices. *Complexity* 2020, 1–10.
- Lu, W., Li, J., Wang, J., Qin, L., 2021. A CNN-BiLSTM-AM method for stock price prediction. *Neural Comput. & Applic.* 33, 4741–4753.
- Lu, Y., Zhang, H., Guo, Q., 2023. Stock and market index prediction using Informer network. *arXiv preprint arXiv:2305.14382*.
- Merello, S., Ratto, A.P., Oneto, L., Cambria, E., 2019. Ensemble application of transfer learning and sample weighting for stock market prediction. In: 2019 International Joint Conference on Neural Networks (IJCNN). pp. 1–8.
- Miau, F., Itti, L., 2001. A neural model combining attentional orienting to object recognition: preliminary explorations on the interplay between where and what. In: 2001 Conference Proceedings of the 23rd Annual International Conference of the IEEE Engineering in Medicine and Biology Society, pp. 789–792.
- Ni, L., Li, Y., Wang, X., Zhang, J., Yu, J., Qi, C., 2019. Forecasting of forex time series data based on deep learning. *Procedia Comput. Sci.* 147, 647–652.
- Pan, S.J., Yang, Q., 2009. A survey on transfer learning. *IEEE Trans. Knowl. Data Eng.* 22 (10), 1345–1359.
- Sawhney, R., Agarwal, S., Wadhwa, A., Shah, R., 2020. Deep attentive learning for stock movement prediction from social media text and company correlations. In: the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 8415–8426.
- Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G., 2008. The graph neural network model. *IEEE Trans. Neural Netw.* 20 (1), 61–80.
- Schmidhuber, J., Huber, R., 1991. Learning to generate artificial fovea trajectories for target detection. *Int. J. Neural Syst.* 2 (01n02), 125–134.
- Shimodaira, H., 2000. Improving predictive inference under covariate shift by weighting the log-likelihood function. *J. Statist. Plann. Inference* 90 (2), 227–244.
- Soydaner, D., 2022. Attention mechanism in neural networks: where it comes from and where it goes. *Neural Comput. & Applic.* 34 (16), 13371–13385.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., Liu, Y., 2024. Roformer: enhanced transformer with rotary position embedding. *Neurocomputing* 568, 127063.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems*, p. 30.
- Wang, S., 2023. A stock price prediction method based on BiLSTM and improved transformer. *IEEE Access* 11, 104211–104223.
- Wang, C., Chen, Y., Zhang, S., Zhang, Q., 2022. Stock market index prediction using deep transformer model. *Expert Syst. Appl.* 208, 118128.
- Weiss, K., Khoshgoftaar, T.M., Wang, D., 2016. A survey of transfer learning. *Journal of Big Data* 3 (1), 1–40.
- Wu, D., Wang, X., Wu, S., 2022. Jointly modeling transfer learning of industrial chain information and deep learning for stock prediction. *Expert Syst. Appl.* 191, 116257.
- Wu, H., Xu, J., Wang, J., Long, M., 2021. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. In: *Advances in Neural Information Processing Systems* 34.
- Xu, C., Huang, H., Ying, X., Gao, J., Li, Z., Zhang, P., Luo, J., 2022. HGNN: hierarchical graph neural network for predicting the classification of price-limit-hitting stocks. *Inf. Sci.* 607, 783–798.
- Yang, L., Zhang, Z., Xiong, S., Wei, L., Ng, J., Xu, L., Dong, R., 2018. Explainable text-driven neural network for stock prediction. In: 2018 5th IEEE International Conference on Cloud Computing and Intelligence Systems (CCIS). pp. 441–445.
- Ying, L., Qian Nan, Z., Fu Ping, W., Tuan Kiang, C., Keng Pang, L., Heng Chang, Z., Nam, L., 2021. Adaptive weights learning in CNN feature fusion for crime scene investigation image classification. *Connect. Sci.* 33 (3), 719–734.
- Zhang, Q., Qin, C., Zhang, Y., Bao, F., Zhang, C., Liu, P., 2022. Transformer-based attention network for stock movement prediction. *Expert Syst. Appl.* 202, 117239.
- Zhang, W., Yang, H., Samaras, D., Zelinsky, G., 2005. A computational model of eye movements during object class detection. In: *Advances in Neural Information Processing Systems* 18.